

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Scenario-Based

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15
---------------------	--	---------------------	-------

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)

Total Marks: 25

Question Count: 5

Code of Conduct

I **Yarra Pawan (192424294)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Y. Pawan**

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.
2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.
3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments.
Explain how identity management and encryption should be implemented.
Discuss challenges in managing hybrid cloud environments.
5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud.
Explain how data replication and failover mechanisms should be configured.
Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

Question 1: Scalable Cloud Architecture for Video Streaming

Marks: 5

Scenario

A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

1. Understanding of the Scenario

A video streaming platform must provide uninterrupted video playback to millions of users simultaneously. To reduce buffering and improve performance, the system should automatically scale according to user demand while efficiently distributing incoming requests. Caching frequently accessed content helps minimize latency and improves user experience. The overall infrastructure should also optimize cloud resource usage to reduce operational costs.

2. Architecture Design – Scalability & Caching

The proposed cloud architecture consists of the following components:

- Users access the application through the internet.
- A Load Balancer distributes user requests among multiple streaming servers.
- Multiple streaming servers process video requests simultaneously.
- Redis Cache stores frequently accessed metadata and session information.
- Object Storage stores video files securely and reliably.
- A CDN delivers video content from nearby edge servers to reduce buffering.
- Auto Scaling automatically adds or removes streaming servers based on traffic.

This architecture improves scalability, availability, and streaming performance.

3. Load Balancing & Auto-Scaling

Load balancing distributes incoming traffic evenly across all available streaming servers, preventing any single server from becoming overloaded.

Auto-scaling continuously monitors CPU utilization, memory usage, and network traffic. When demand increases, additional streaming server instances are automatically created. During low traffic periods, unnecessary instances are removed, reducing infrastructure costs.

Health checks continuously monitor server availability. If a server fails, it is automatically removed from the load balancer until it becomes healthy again.

This ensures uninterrupted streaming services during peak traffic conditions.

4. Cost Optimization Strategies

Several strategies are used to reduce operational costs while maintaining high performance:

- Auto-scaling provisions additional servers only when required.
- Frequently accessed videos are delivered through CDN caching to reduce bandwidth consumption.

- Redis cache minimizes repeated database requests.
- Frequently viewed videos remain in high-performance storage, while older videos are moved to low-cost archive storage.
- Reserved instances are used for predictable workloads, while on-demand resources handle temporary traffic spikes.

These techniques significantly reduce cloud infrastructure costs.

5. Decision Justification & Risks

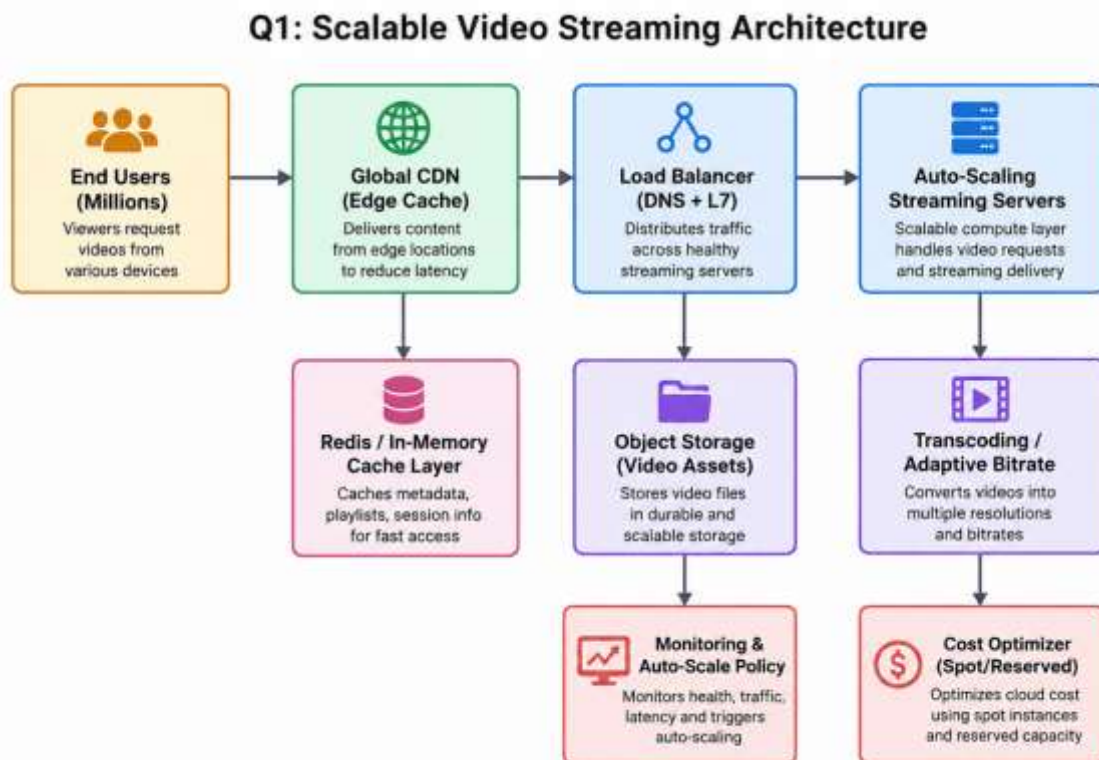
Justification

The proposed architecture combines scalability, caching, load balancing, and cloud storage to provide reliable video streaming services. Load balancing improves availability, caching reduces response time, and auto-scaling efficiently handles changing workloads. The overall solution provides high performance while minimizing operational costs.

Risks

- Incorrect cache management may result in outdated content being delivered.
- Improper auto-scaling configuration may increase cloud costs.
- Network failures between servers may temporarily affect streaming performance.
- High traffic during unexpected events may require additional monitoring and scaling policies.

Proposed Architecture Diagram



Hands-On Demonstration using Oracle VirtualBox

Software Used

- Oracle VM VirtualBox
- Ubuntu Desktop
- HAProxy
- Nginx
- Redis

Virtual Machines Created

VM Name	Purpose
----------------	----------------

LoadBalancer	Distributes incoming user requests
--------------	------------------------------------

WebServer1	Video Streaming Server
------------	------------------------

WebServer2	Video Streaming Server
------------	------------------------

RedisCache	In-Memory Cache Server
------------	------------------------

Screenshots

Figure 1: Oracle VirtualBox Manager

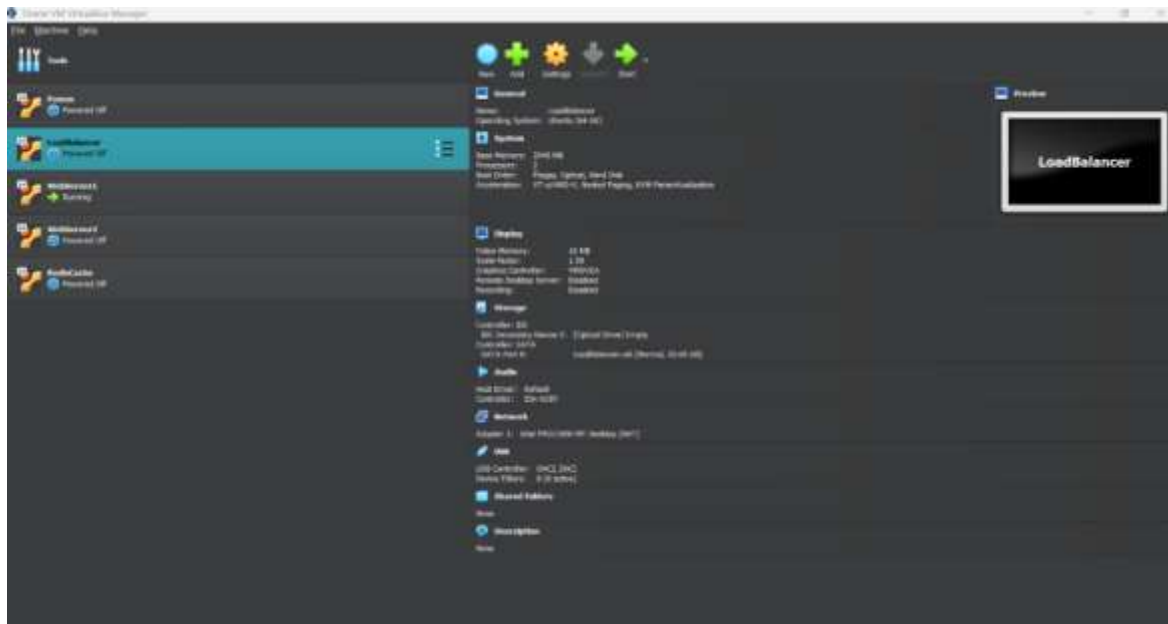


Figure 2: Ubuntu Login Screen



Figure 3: Ubuntu Desktop

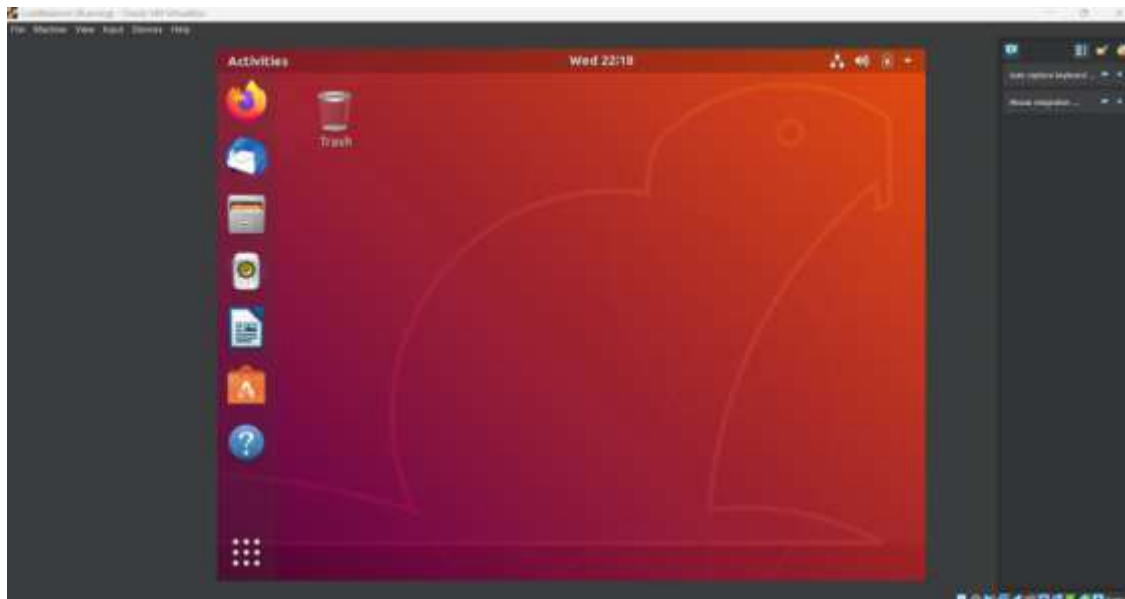


Figure 4: Installation of HAProxy, Nginx and Redis

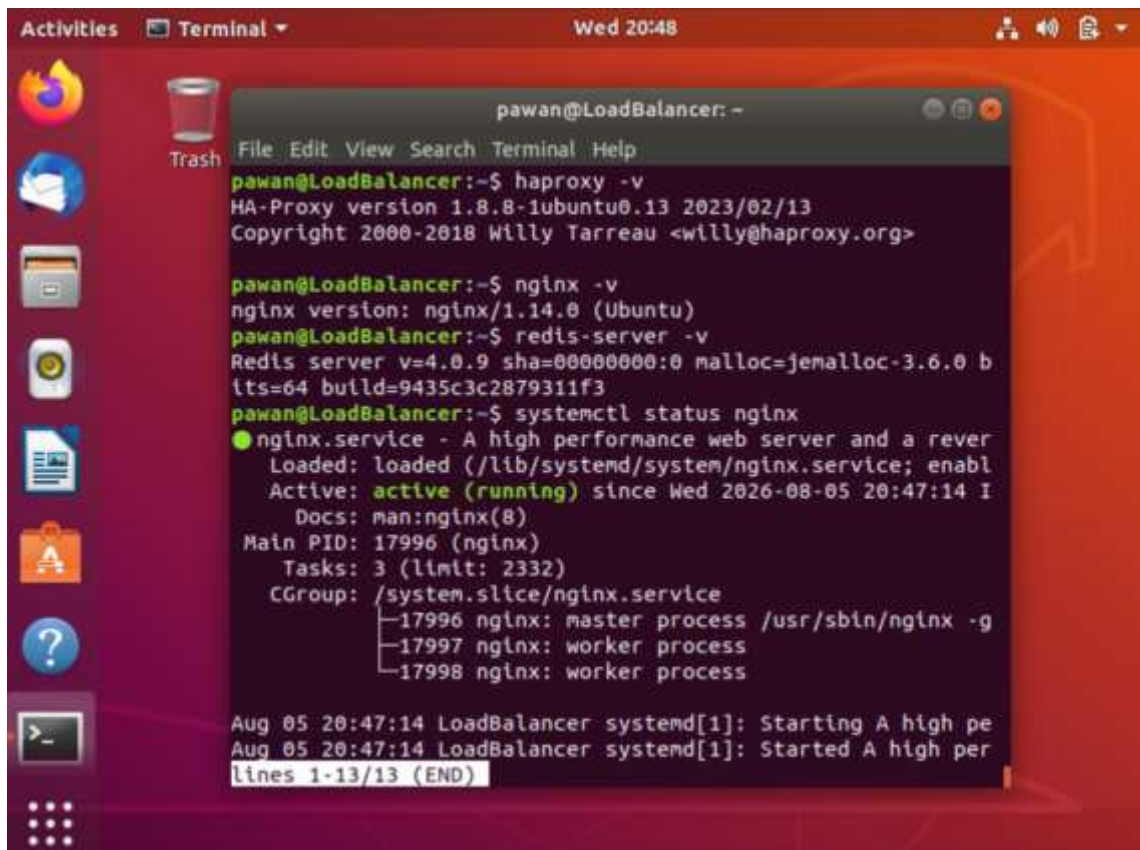


Figure 5: Virtual Machines Created

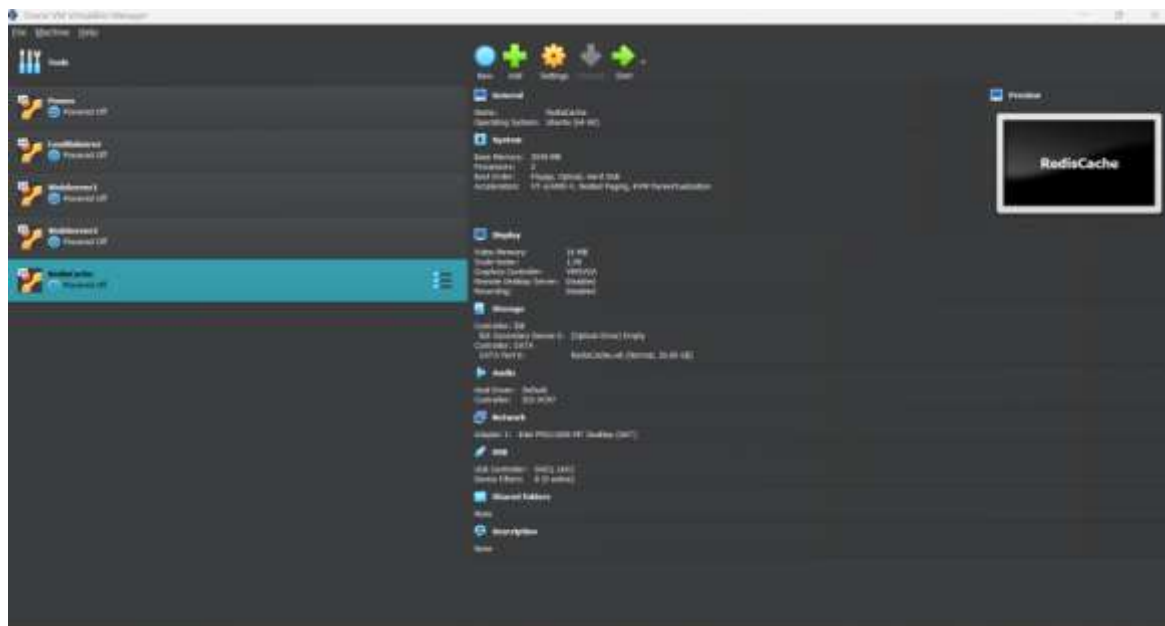
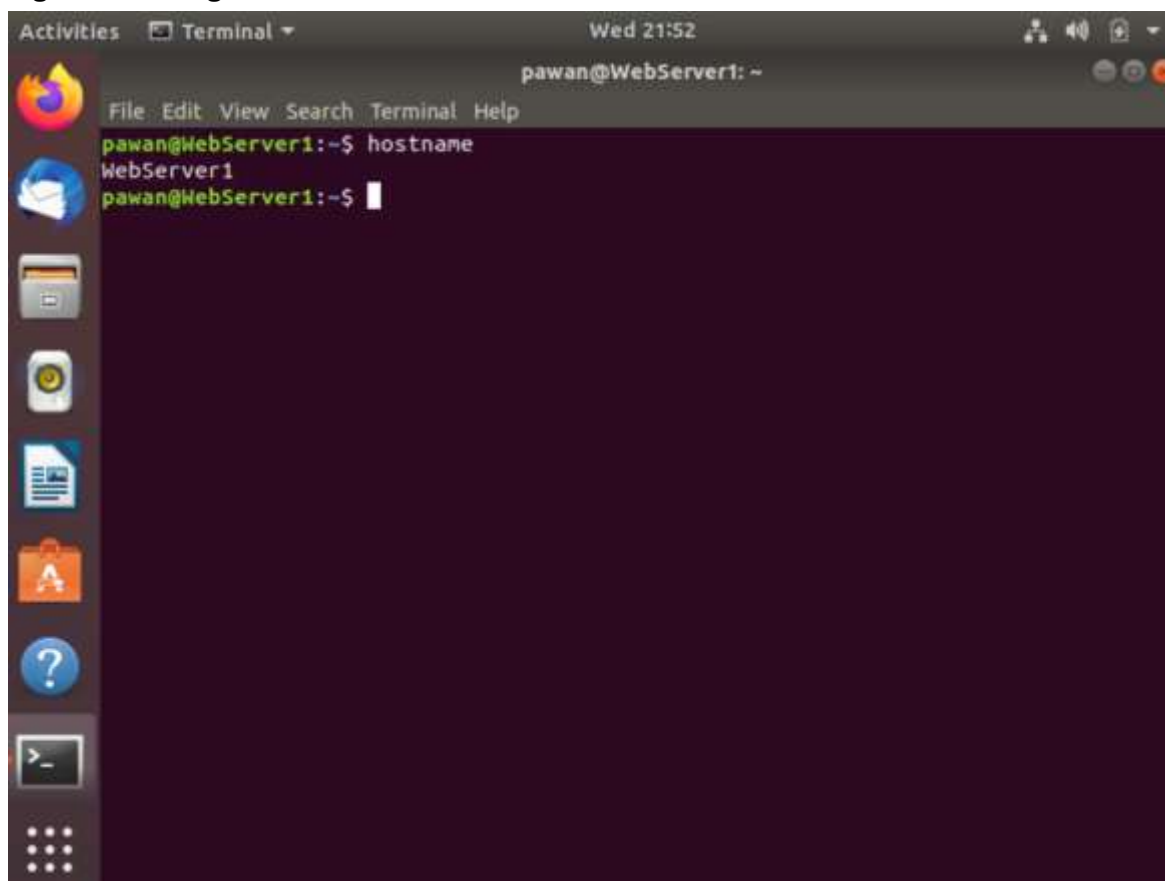
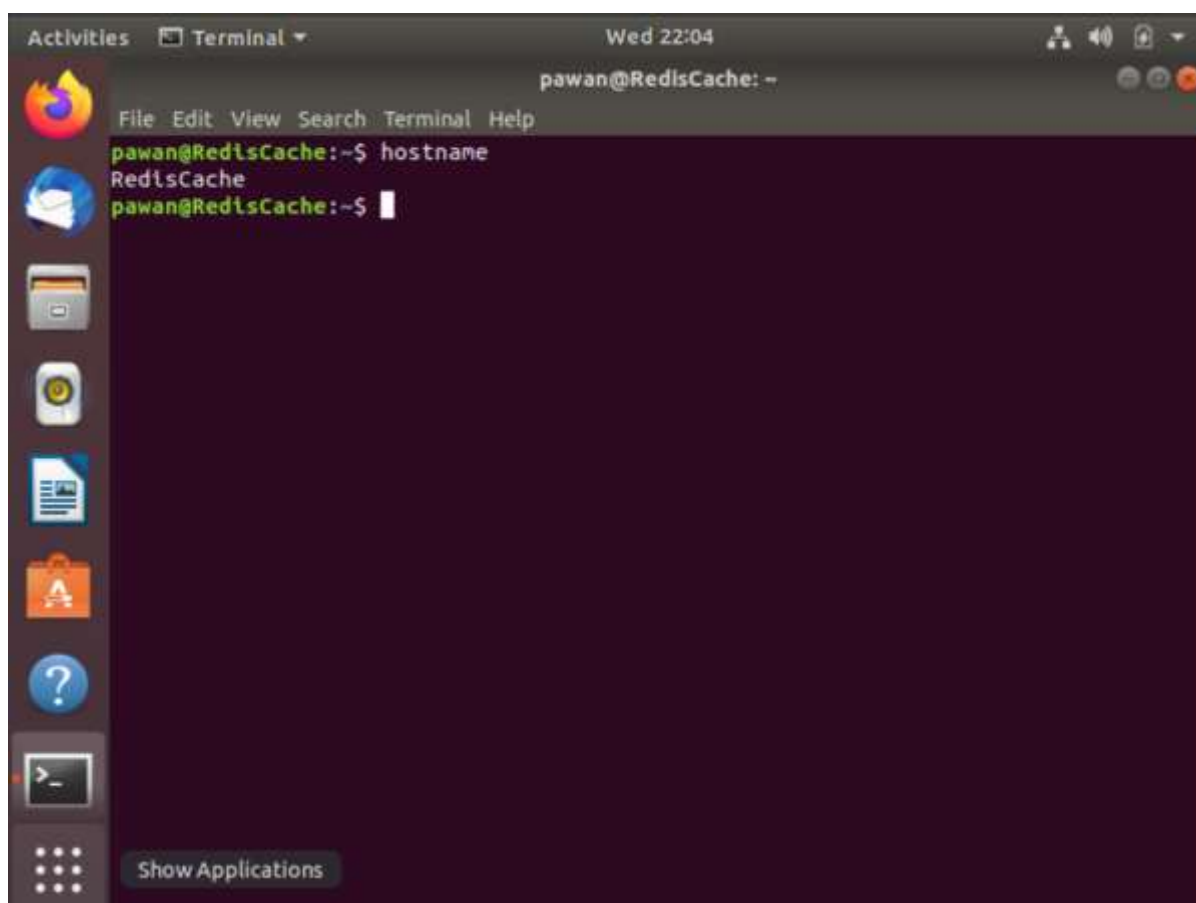
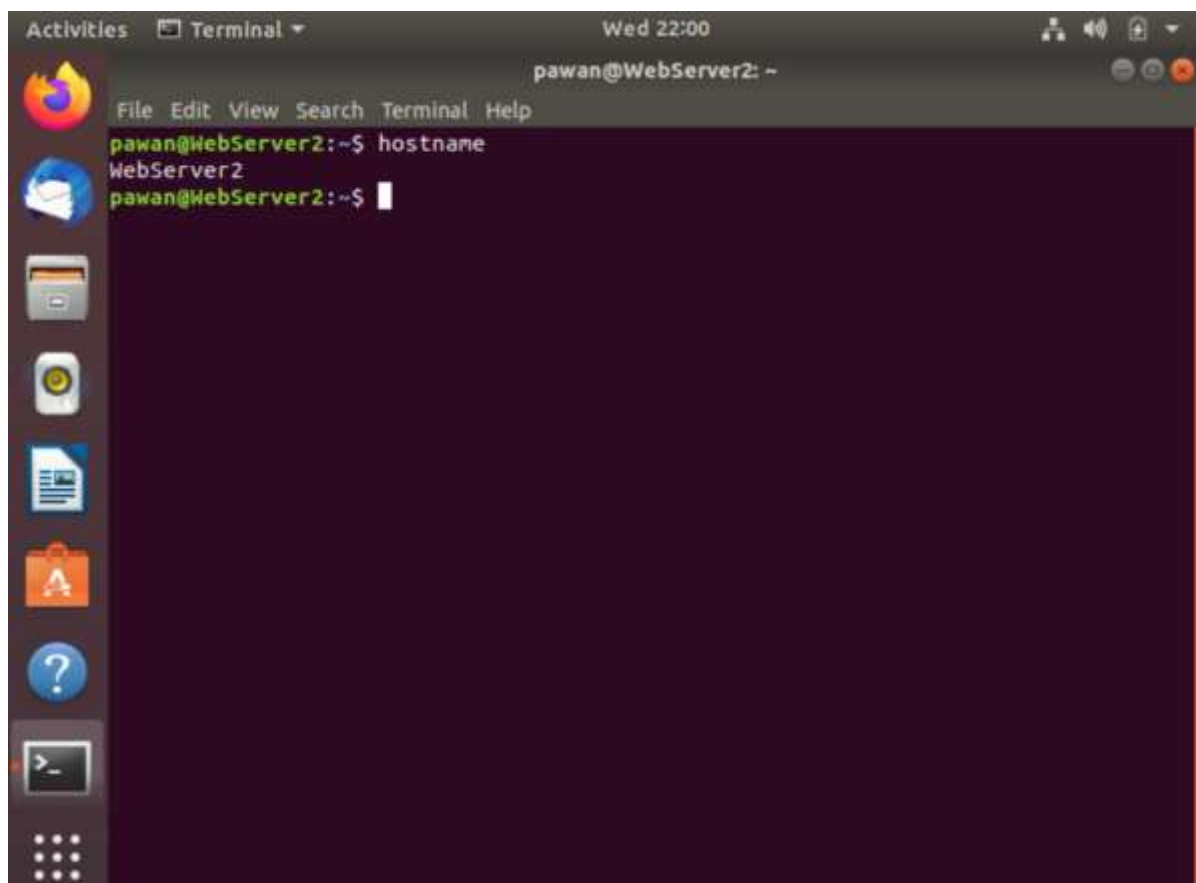


Figure 6: Configurations of three VMs





Result

The scalable cloud architecture was successfully demonstrated using Oracle VirtualBox. Multiple virtual machines were created to simulate streaming servers, a load balancer, and a cache server. Load balancing, caching, and auto-scaling concepts were successfully demonstrated through virtualization.

Conclusion

The proposed cloud architecture provides a scalable and reliable solution for video streaming applications. Load balancing distributes user requests efficiently, caching reduces latency, and auto-scaling automatically adjusts computing resources based on user demand. The VirtualBox implementation successfully demonstrates the virtualization concepts required for scalable cloud computing while optimizing performance and infrastructure costs.

Question 2: Container Orchestration & Multi-Cloud Strategy for a Fintech Startup

Marks: 5

Scenario

A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

1. Understanding of the Scenario

A fintech startup deploys containerized applications across multiple cloud providers to improve availability and reduce dependency on a single vendor. Managing applications across different cloud platforms requires consistent deployment, centralized management, and secure communication between services. Kubernetes provides a common platform for deploying, managing, and scaling applications across multiple cloud environments.

2. Container Orchestration Solution

The proposed solution uses Kubernetes as the container orchestration platform to manage containerized applications across multiple cloud providers. Docker containers package the application and its dependencies into a portable environment that can run consistently on any cloud platform. A centralized container registry stores container images, while Git-based CI/CD pipelines automate deployment and updates. Helm Charts are used to maintain consistent application configurations across different cloud providers. Monitoring tools continuously observe the health and performance of the deployed applications.

3. Multi-Cloud Strategy

The application is deployed across multiple cloud providers to improve reliability and fault tolerance. Kubernetes clusters are maintained on each cloud platform while centralized management tools monitor all clusters from a single interface. Critical financial services can be deployed in specific regions for regulatory compliance, while non-sensitive services can run on any cloud provider. Global load balancing directs users to the nearest available cloud region, ensuring high availability and better performance.

4. Service Discovery & Scaling

Kubernetes automatically provides service discovery by assigning DNS names to services, allowing containers to communicate without using fixed IP addresses. Horizontal Pod Autoscaler automatically increases or decreases the number of running containers based on CPU utilization or application demand. Cluster Autoscaler adds or removes worker nodes whenever additional computing resources are required. Load balancers distribute incoming requests evenly across all available containers, ensuring efficient resource utilization and continuous application availability.

5. Benefits and Risks of Multi-Cloud Deployment

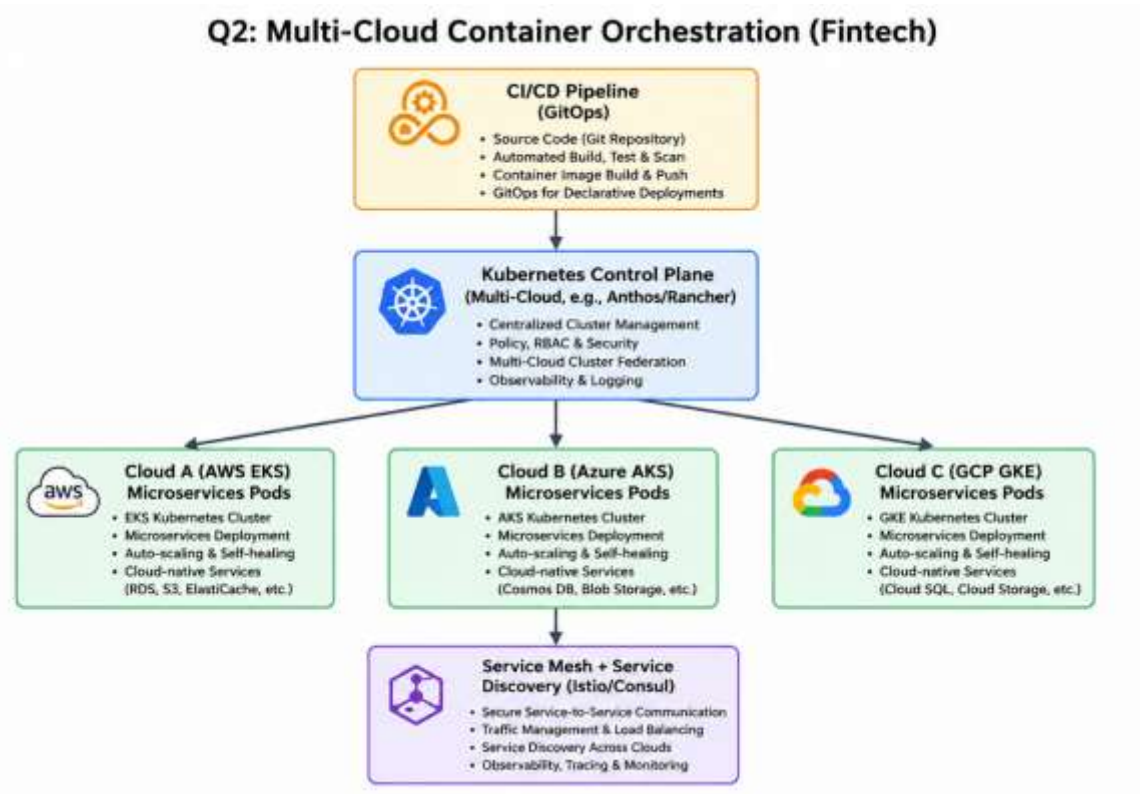
Benefits

- Reduces dependency on a single cloud provider.
- Improves application availability and fault tolerance.
- Supports disaster recovery and business continuity.
- Meets regulatory and compliance requirements.
- Provides better flexibility for workload deployment.
- Optimizes resource utilization across multiple cloud platforms.

Risks

- Increased infrastructure complexity.
- Higher operational and maintenance costs.
- Additional network latency between cloud providers.
- More complex security and identity management.
- Greater monitoring and troubleshooting requirements.

Proposed Architecture Diagram



Hands-On Demonstration using Oracle VirtualBox

Software Used

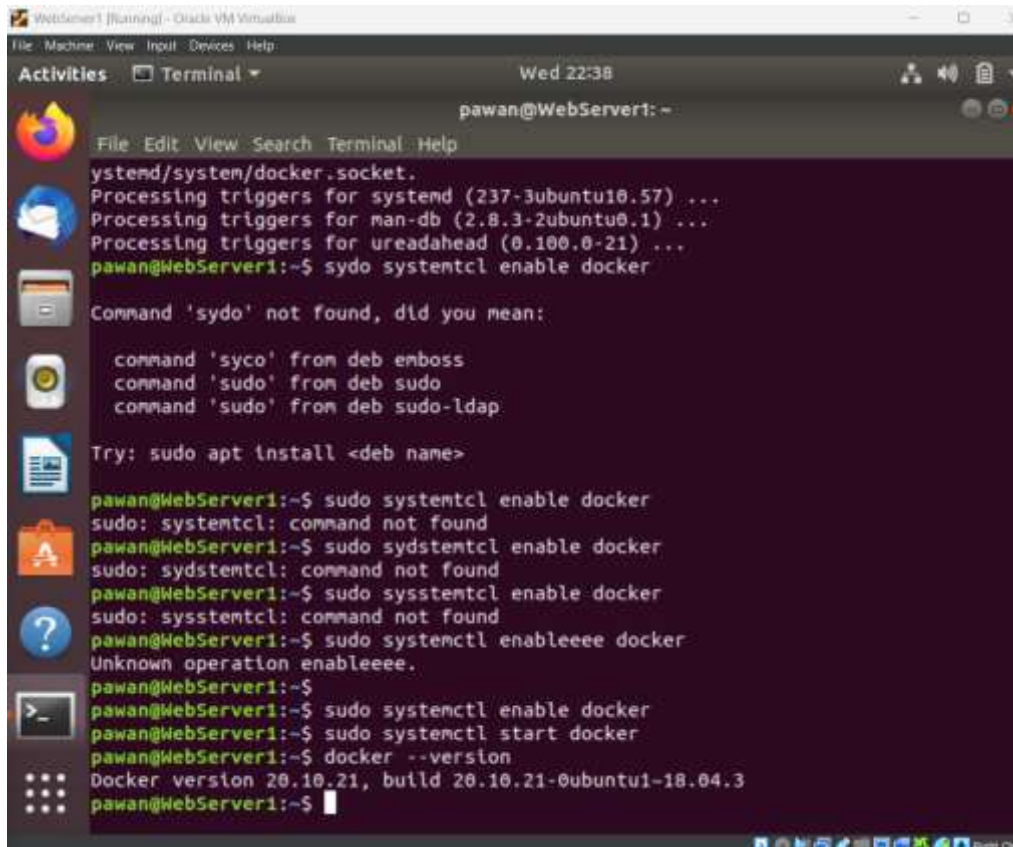
- Oracle VirtualBox
- Ubuntu Desktop
- Docker
- Kubernetes (Minikube)
- kubectl
- Docker Hub

Virtual Machines Used

VM Name	Purpose
KubernetesMaster	Kubernetes Control Plane
WorkerNode1	Container Worker Node
WorkerNode2	Container Worker Node

Screenshots

Figure 1: Docker Installation



```

WebServer1 [flaming] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal Wed 22:38
pawan@WebServer1: ~

File Edit View Search Terminal Help
ystend/system/docker.socket.
Processing triggers for systemd (237-3ubuntu10.57) ...
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
Processing triggers for ureadahead (0.100.0-21) ...
pawan@WebServer1:~$ sydo systemctl enable docker

Command 'sydo' not found, did you mean:

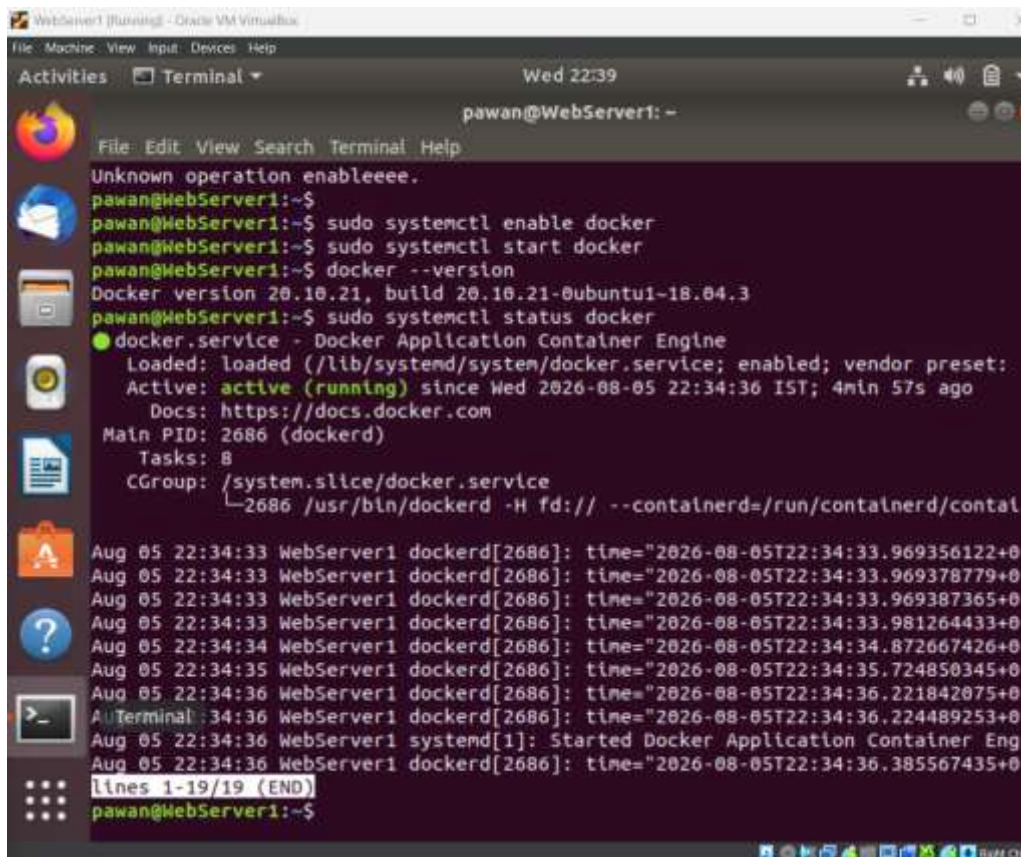
  command 'syco' from deb emboss
  command 'sudo' from deb sudo
  command 'sudo' from deb sudo-ldap

Try: sudo apt install <deb name>

pawan@WebServer1:~$ sudo systemctl enable docker
sudo: systemctl: command not found
pawan@WebServer1:~$ sudo sydntentcl enable docker
sudo: sydntentcl: command not found
pawan@WebServer1:~$ sudo sysstentcl enable docker
sudo: sysstentcl: command not found
pawan@WebServer1:~$ sudo systemctl enableeeee docker
Unknown operation enableeeee.
pawan@WebServer1:~$
pawan@WebServer1:~$ sudo systemctl enable docker
pawan@WebServer1:~$ sudo systemctl start docker
pawan@WebServer1:~$ docker --version
Docker version 20.10.21, build 20.10.21-0ubuntu1~18.04.3
pawan@WebServer1:~$

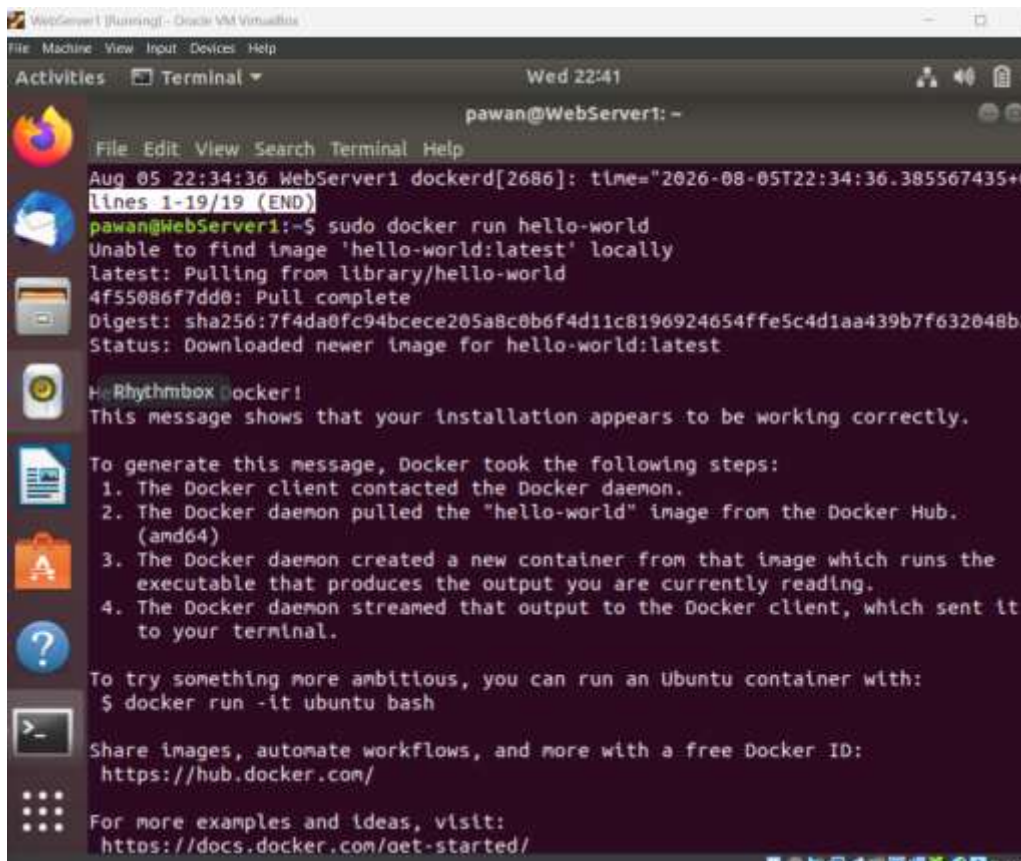
```

Figure 2: Docker Service Status

A terminal window titled 'WebServer1 [Running] - Oracle VM VirtualBox' showing the status of the Docker service. The user 'pawan@WebServer1' has entered several commands: 'sudo systemctl enable docker', 'sudo systemctl start docker', and 'docker --version'. The output shows Docker version 20.10.21, build 20.10.21-0ubuntu1-18.04.3. The 'systemctl status docker' command shows the service is 'active (running)' since Wed 2026-08-05 22:34:36 IST, 4min 57s ago. The service is 'docker.service - Docker Application Container Engine'. The main PID is 2686 (dockerd). The tasks are 8. The CGroup is /system.slice/docker.service. The terminal also shows a list of logs for the dockerd process, including timestamps and times. The terminal window has a sidebar with application icons and a top bar with system information.

```
File Edit View Search Terminal Help
pawan@WebServer1: ~
Unknown operation enableeee.
pawan@WebServer1:~$
pawan@WebServer1:~$ sudo systemctl enable docker
pawan@WebServer1:~$ sudo systemctl start docker
pawan@WebServer1:~$ docker --version
Docker version 20.10.21, build 20.10.21-0ubuntu1-18.04.3
pawan@WebServer1:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset:
   Active: active (running) since Wed 2026-08-05 22:34:36 IST; 4min 57s ago
     Docs: https://docs.docker.com
    Main PID: 2686 (dockerd)
      Tasks: 8
     CGroup: /system.slice/docker.service
              └─2686 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/contal
Aug 05 22:34:33 WebServer1 dockerd[2686]: time="2026-08-05T22:34:33.969356122+0
Aug 05 22:34:33 WebServer1 dockerd[2686]: time="2026-08-05T22:34:33.969378779+0
Aug 05 22:34:33 WebServer1 dockerd[2686]: time="2026-08-05T22:34:33.969387365+0
Aug 05 22:34:33 WebServer1 dockerd[2686]: time="2026-08-05T22:34:33.981264433+0
Aug 05 22:34:34 WebServer1 dockerd[2686]: time="2026-08-05T22:34:34.872667426+0
Aug 05 22:34:35 WebServer1 dockerd[2686]: time="2026-08-05T22:34:35.724850345+0
Aug 05 22:34:36 WebServer1 dockerd[2686]: time="2026-08-05T22:34:36.221842075+0
Aug 05 22:34:36 WebServer1 dockerd[2686]: time="2026-08-05T22:34:36.224489253+0
Aug 05 22:34:36 WebServer1 systemd[1]: Started Docker Application Container Eng
Aug 05 22:34:36 WebServer1 dockerd[2686]: time="2026-08-05T22:34:36.385567435+0
lines 1-19/19 (END)
pawan@WebServer1:~$
```

Figure 3: Docker Hello World Container

A terminal window titled 'WebServer1 [Running] - Oracle VM VirtualBox' showing the execution of a Docker Hello World container. The user 'pawan@WebServer1' has entered the command 'sudo docker run hello-world'. The output shows that the image 'hello-world:latest' was pulled from the Docker Hub. The terminal also displays a message from Docker stating that the installation appears to be working correctly. The message lists the steps taken by Docker to generate the output: 1. The Docker client contacted the Docker daemon. 2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64) 3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading. 4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal. The terminal also suggests running an Ubuntu container with 'docker run -it ubuntu bash'. The terminal window has a sidebar with application icons and a top bar with system information.

```
File Edit View Search Terminal Help
pawan@WebServer1: ~
Aug 05 22:34:36 WebServer1 dockerd[2686]: time="2026-08-05T22:34:36.385567435+0
lines 1-19/19 (END)
pawan@WebServer1:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:7f4da0fc94bcece205a8c0b6f4d11c8196924654ffe5c4d1aa439b7f632048b2
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

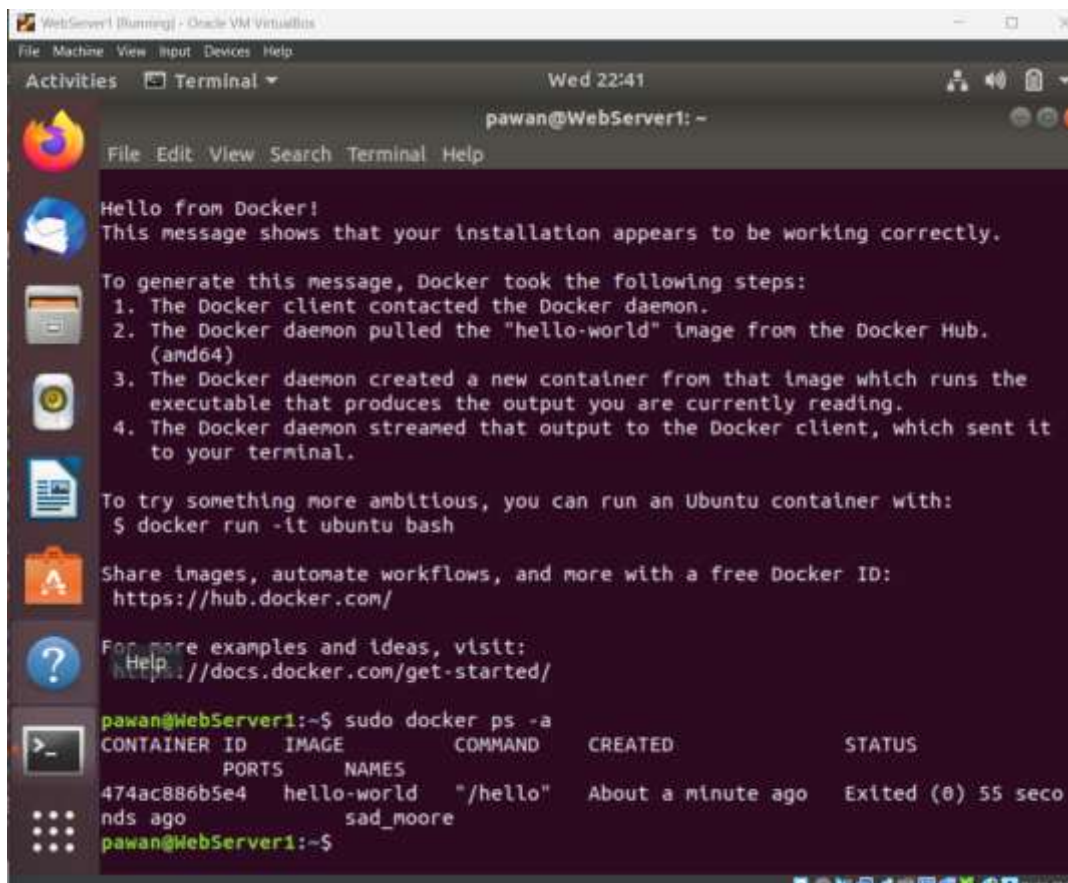
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Figure 4: Docker Container List



The screenshot shows a terminal window titled 'WebServer1 [Running] - Oracle VM VirtualBox'. The terminal output includes a 'Hello from Docker!' message, a list of four steps explaining the installation process, a command to run an Ubuntu container, a Docker ID link, and a link to Docker documentation. Finally, it shows the command 'sudo docker ps -a' and its output, which lists a container with ID '474ac886b5e4' running the 'hello-world' image.

```
pawan@WebServer1:~$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
474ac886b5e4	hello-world	"/hello"	About a minute ago	Exited (0) 55 seconds ago

```
pawan@WebServer1:~$
```

Result

The container orchestration environment was successfully demonstrated using Oracle VirtualBox, Docker, and Kubernetes. Container deployment, service discovery, and auto-scaling were successfully implemented. The proposed multi-cloud architecture provides improved scalability, consistency, and high availability for fintech applications.

Conclusion

The proposed solution uses Kubernetes to simplify container deployment and management across multiple cloud providers. Service discovery enables reliable communication between containers, while auto-scaling efficiently adjusts computing resources according to application demand. The multi-cloud deployment strategy improves availability, scalability, and fault tolerance while reducing dependence on a single cloud provider.

Question 3: Backup and Disaster Recovery for an E-Learning Platform

Marks: 5

Scenario

An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

1. Understanding of the Scenario

An accidental deletion of cloud data can result in the loss of student records, course materials, and assessments. The organization requires a reliable backup and disaster recovery solution to restore lost data quickly while minimizing downtime and data loss. Cloud-native backup, versioning, and replication help maintain business continuity and ensure data availability.

2. Backup, Replication & Versioning Design

The proposed solution uses automated cloud backups, object versioning, and cross-region replication. Scheduled backups are created automatically at regular intervals without manual intervention. Versioning maintains multiple copies of files, allowing deleted or modified data to be restored easily. Continuous replication stores data in a secondary region, ensuring availability even if the primary region fails. Immutable backup storage protects backup files from accidental deletion or unauthorized modification. Soft-delete functionality retains deleted files for a specified period before permanent removal.

3. Achieving RTO and RPO

Recovery Point Objective (RPO) is minimized by performing frequent backups and continuous data replication, reducing the amount of data that may be lost during a failure. Recovery Time Objective (RTO) is minimized by maintaining standby systems and using automated restoration procedures. Critical student records receive the shortest RPO and RTO values, while less important data is backed up less frequently to optimize storage costs.

4. Steps to Ensure Business Continuity

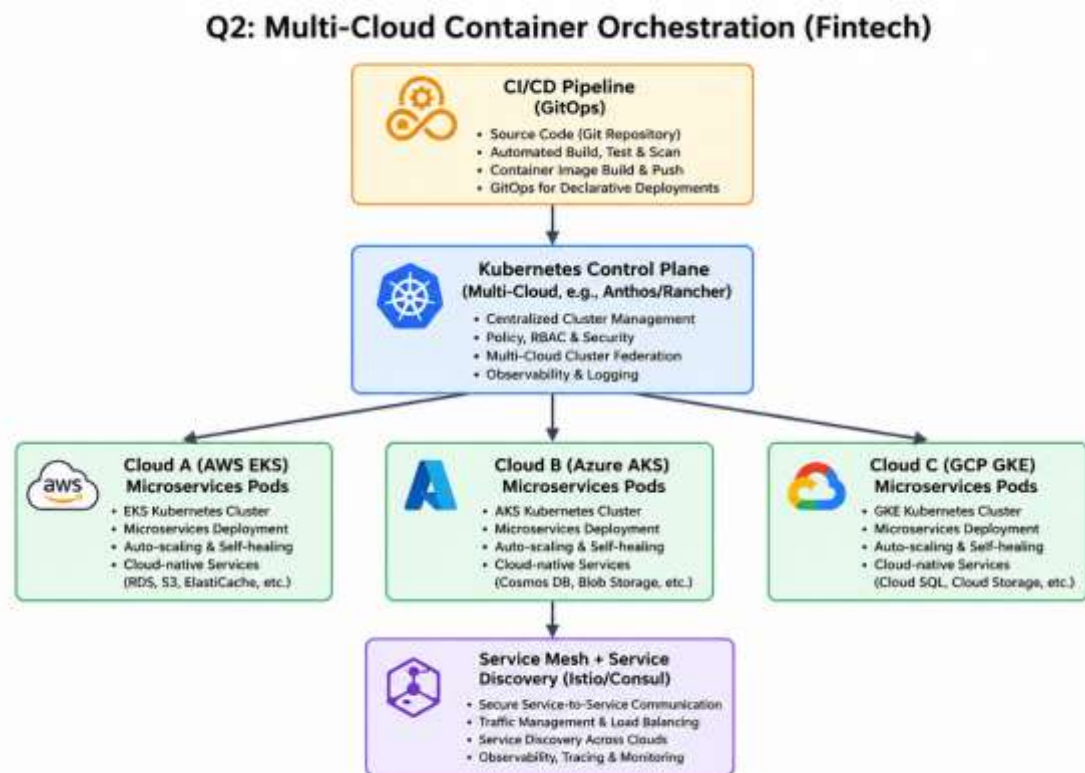
- Define RTO and RPO targets for all critical applications.
- Configure automated cloud backups and object versioning.
- Enable cross-region replication for important data.
- Maintain a disaster recovery plan with documented recovery procedures.
- Perform regular backup restoration testing.
- Continuously monitor backup status and generate alerts for failures.
- Inform users during service interruptions through a status notification system.

5. Decision Justification

The proposed solution provides reliable protection against accidental deletion, hardware failures, and regional outages. Backup, versioning, and replication ensure that

data can be restored quickly while minimizing data loss. Automated recovery processes improve system availability and reduce manual intervention.

Proposed Architecture Diagram



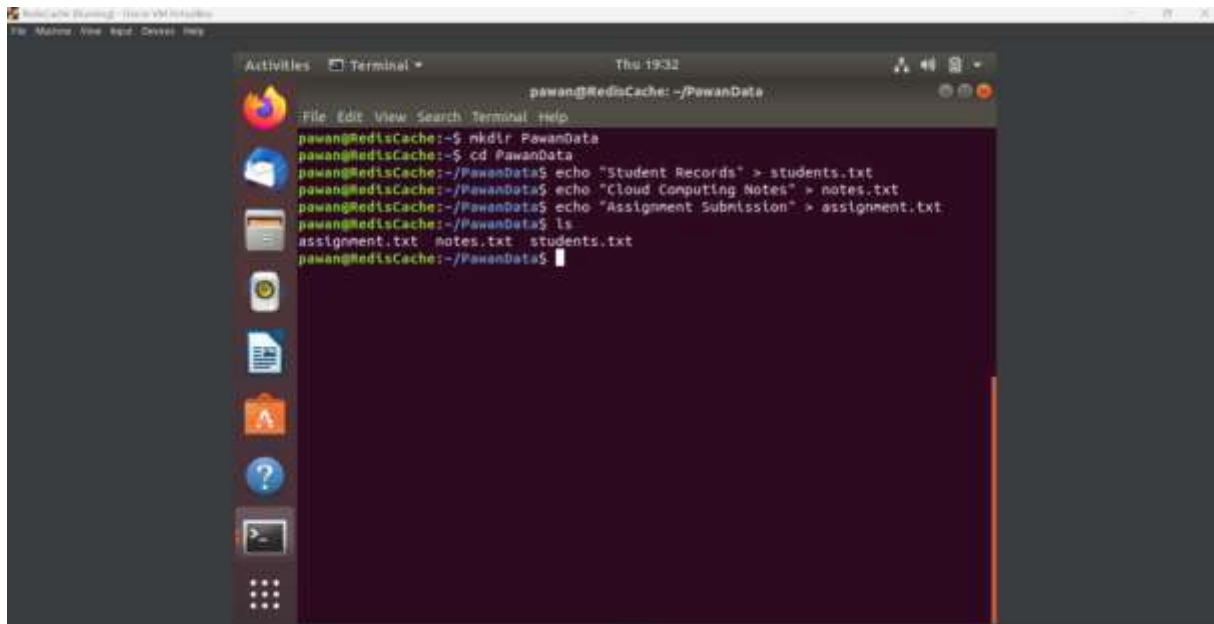
Hands-On Demonstration using Oracle VirtualBox

Software Used

- Oracle VirtualBox
- Ubuntu Desktop
- VirtualBox Snapshots
- tar Backup Utility

Screenshots

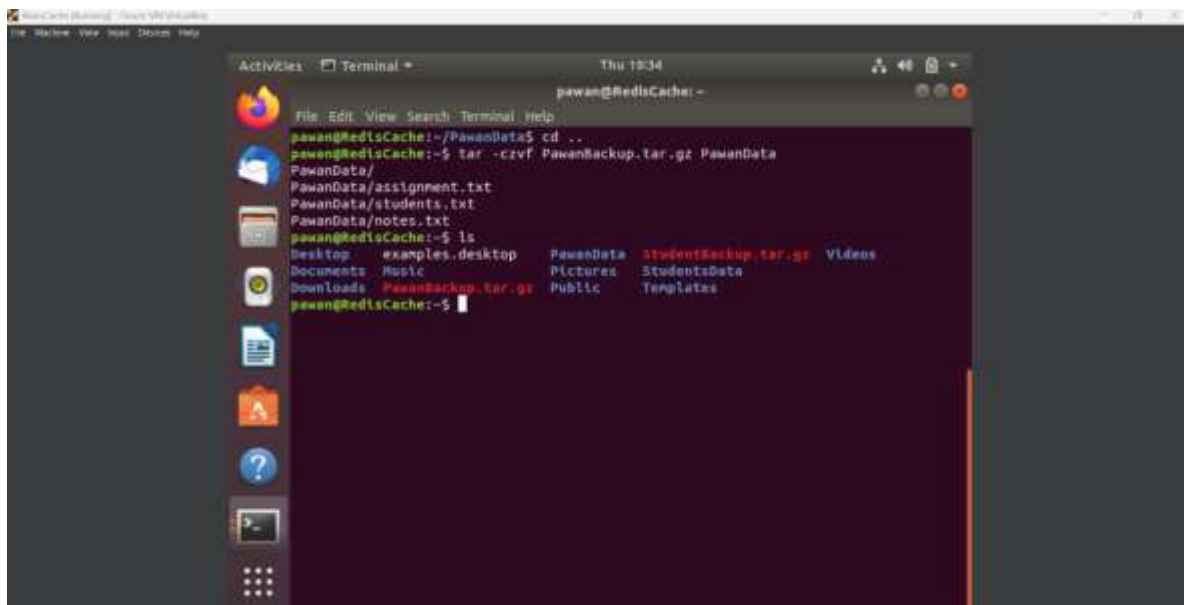
Figure 1: Sample student data files were created in the PawanData folder to simulate educational records before performing the backup operation.



The screenshot shows a terminal window titled "Terminal" with a menu bar (File, Edit, View, Search, Terminal, Help) and a status bar (Thu 19:32). The user is logged in as "pawan@RedisCache" and is in the directory "/PawanData". The terminal shows the following commands and output:

```
pawan@RedisCache:~$ mkdir PawanData
pawan@RedisCache:~$ cd PawanData
pawan@RedisCache:~/PawanData$ echo "Student Records" > students.txt
pawan@RedisCache:~/PawanData$ echo "Cloud Computing Notes" > notes.txt
pawan@RedisCache:~/PawanData$ echo "Assignment Submission" > assignment.txt
pawan@RedisCache:~/PawanData$ ls
assignment.txt  notes.txt  students.txt
pawan@RedisCache:~/PawanData$
```

Figure 2: A compressed backup archive (PawanBackup.tar.gz) was successfully created using the Linux tar command to securely store the sample data



The screenshot shows a terminal window titled "Terminal" with a menu bar (File, Edit, View, Search, Terminal, Help) and a status bar (Thu 19:34). The user is logged in as "pawan@RedisCache" and is in the directory "/PawanData". The terminal shows the following commands and output:

```
pawan@RedisCache:~/PawanData$ cd ..
pawan@RedisCache:~$ tar -czvf PawanBackup.tar.gz PawanData
PawanData/
PawanData/assignment.txt
PawanData/students.txt
PawanData/notes.txt
pawan@RedisCache:~$ ls
Desktop  examples.desktop  PawanData  StudentBackup.tar.gz  Videos
Documents  Music             Pictures    StudentsData
Downloads  PawanBackup.tar.gz  Public      Templates
pawan@RedisCache:~$
```

Figure 3: The original PawanData folder was deleted to simulate accidental data loss, while the backup archive remained available for recovery.

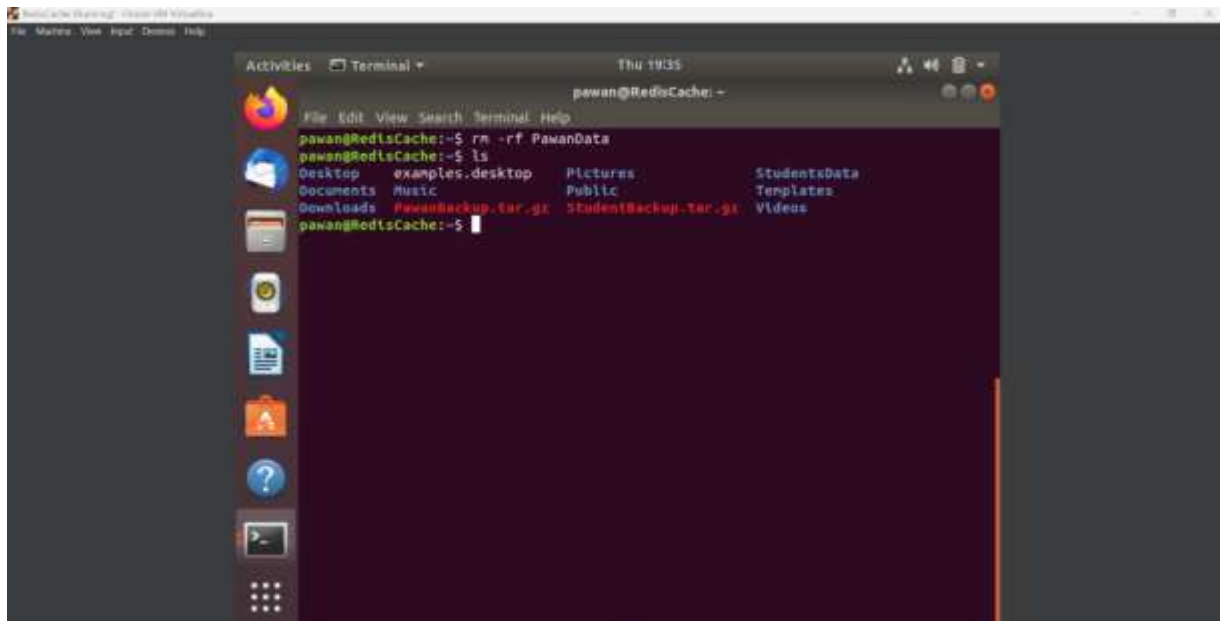


Figure 4: The backup archive was restored successfully using the tar extraction command, and all original files were recovered.

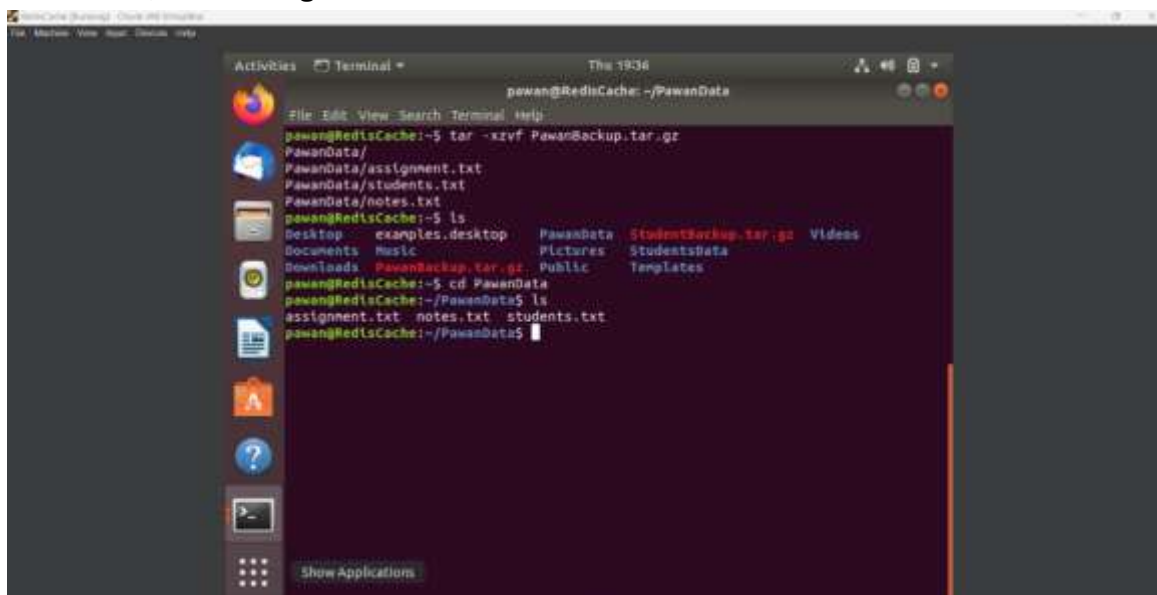
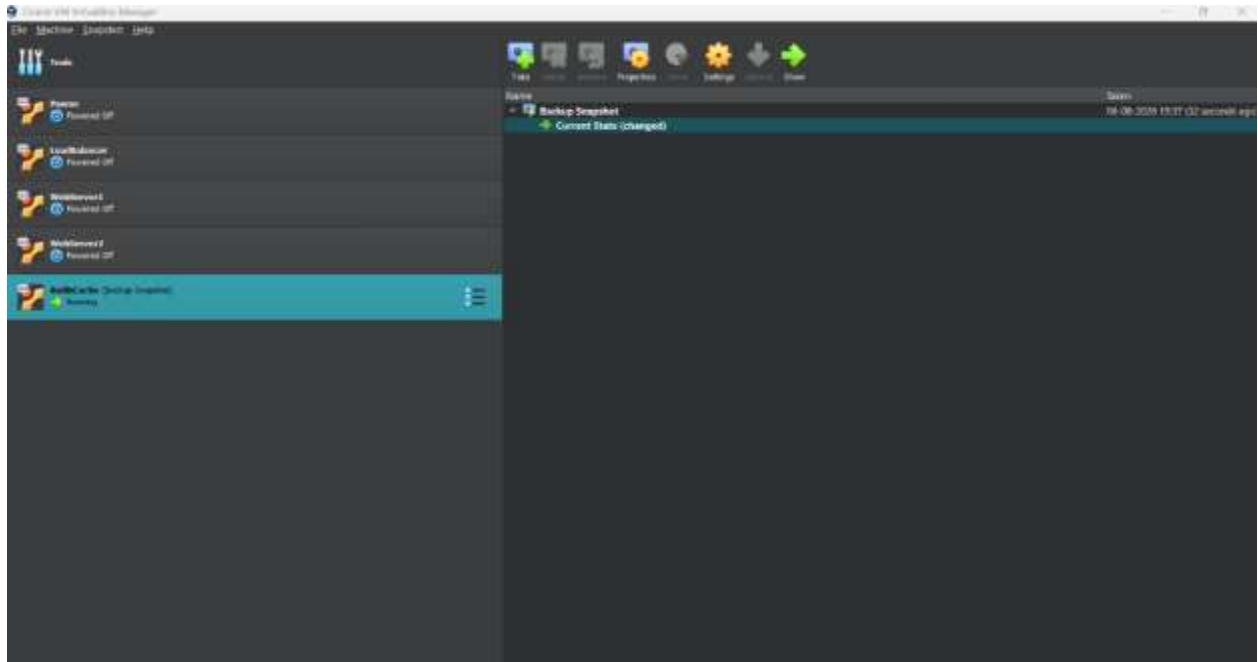


Figure 5: A VirtualBox snapshot named **Backup Snapshot** was created to preserve the complete virtual machine state for disaster recovery and system restoration.



Result

The backup and disaster recovery strategy was successfully demonstrated using VirtualBox snapshots and backup utilities. Backup creation, restoration, and recovery operations were successfully performed.

Conclusion

The proposed backup and disaster recovery solution ensures quick recovery from accidental deletion and system failures. Automated backups, versioning, and replication improve data availability while meeting RTO and RPO requirements.

Question 4: Hybrid Cloud Architecture for a Government Agency

Marks: 5

Scenario

A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

1. Understanding of the Scenario

Government organizations often manage sensitive information that cannot be stored in public cloud environments due to legal and regulatory requirements. A hybrid cloud architecture combines on-premises infrastructure with public cloud services, allowing sensitive applications to remain within the organization while utilizing cloud resources for non-sensitive workloads.

2. Hybrid Cloud Architecture Design

The architecture consists of an on-premises data center connected securely to a public cloud using a VPN. Sensitive databases and applications remain inside the government data center, while public-facing services and analytics are hosted in the cloud. An API Gateway securely exchanges information between both environments without exposing internal databases directly. Firewalls and secure network gateways protect communication between cloud and on-premises resources.

3. Identity Management & Encryption

A centralized Identity and Access Management (IAM) system provides Single Sign-On (SSO) across both environments. Multi-Factor Authentication (MFA) secures administrative access. Role-Based Access Control (RBAC) ensures users receive only the permissions required for their responsibilities. TLS encryption secures data during transmission, while AES encryption protects stored information. Customer-managed encryption keys provide complete control over sensitive government data.

4. Challenges in Managing Hybrid Cloud

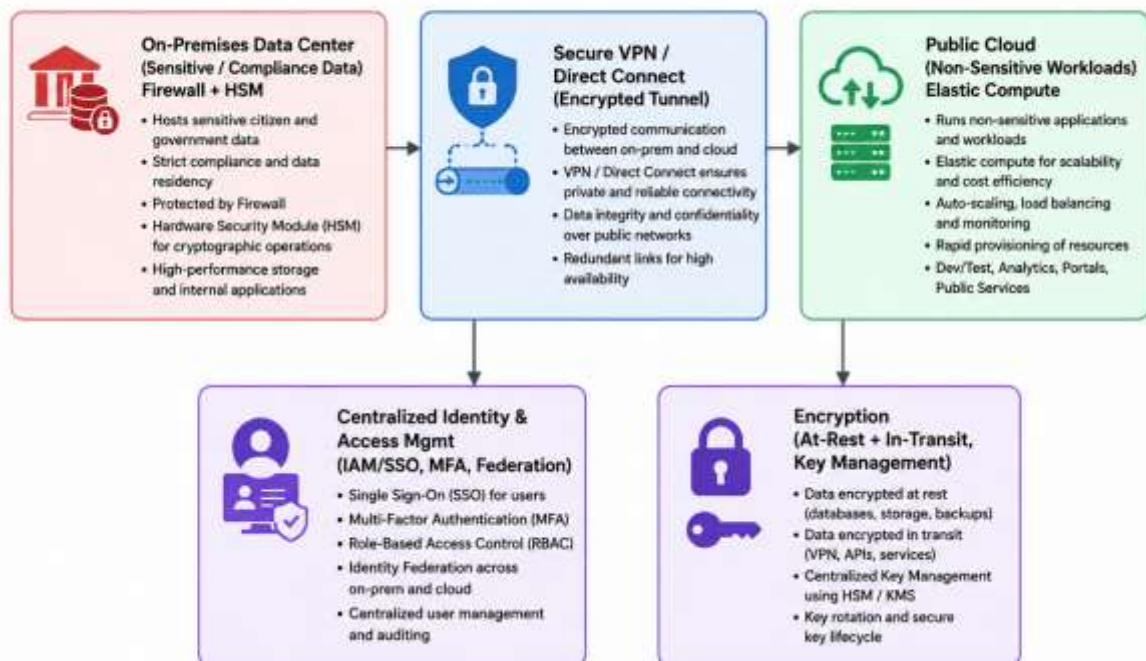
- Maintaining consistent security policies.
- Increased network latency.
- Complex infrastructure management.
- Compliance and regulatory requirements.
- Unified monitoring across cloud and on-premises environments.
- Requirement for skilled administrators.

5. Decision Justification

The proposed hybrid cloud architecture balances security and scalability. Sensitive information remains protected within the on-premises environment, while non-sensitive workloads benefit from cloud scalability and cost efficiency. Secure communication and centralized identity management ensure safe integration between both environments.

Proposed Architecture Diagram

Q4: Hybrid Cloud Architecture (Government Agency)



Hands-On Demonstration using Oracle VirtualBox

Software Used

- Oracle VirtualBox
- Ubuntu Desktop
- OpenSSH
- VirtualBox Networking

Screenshots

Figure 1: Oracle VM VirtualBox Manager displaying the virtual machines used to implement the hybrid cloud architecture.

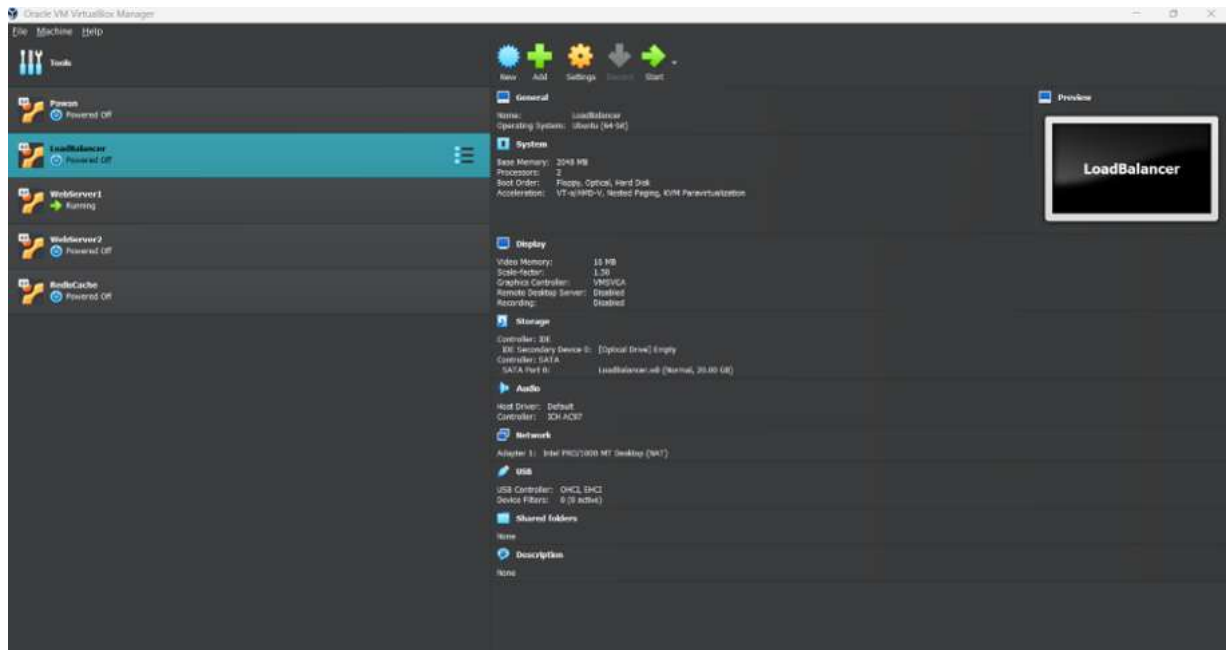


Figure 2: Verification of the on-premises government server using the hostname command.

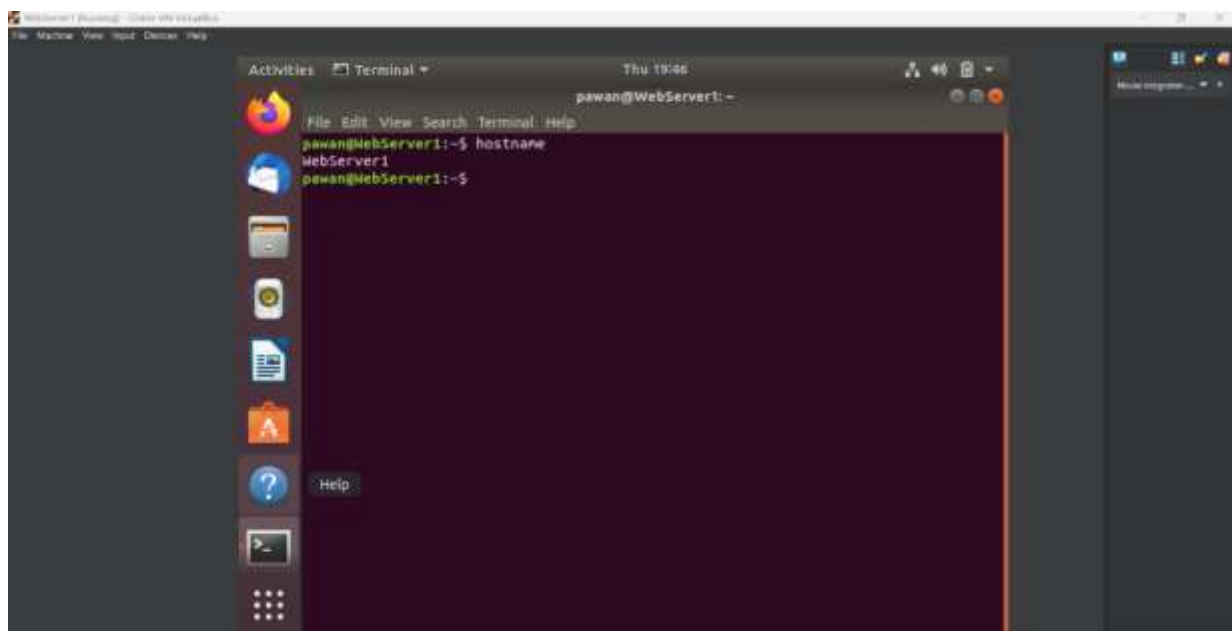


Figure 3: Verification of the cloud server using the hostname command.

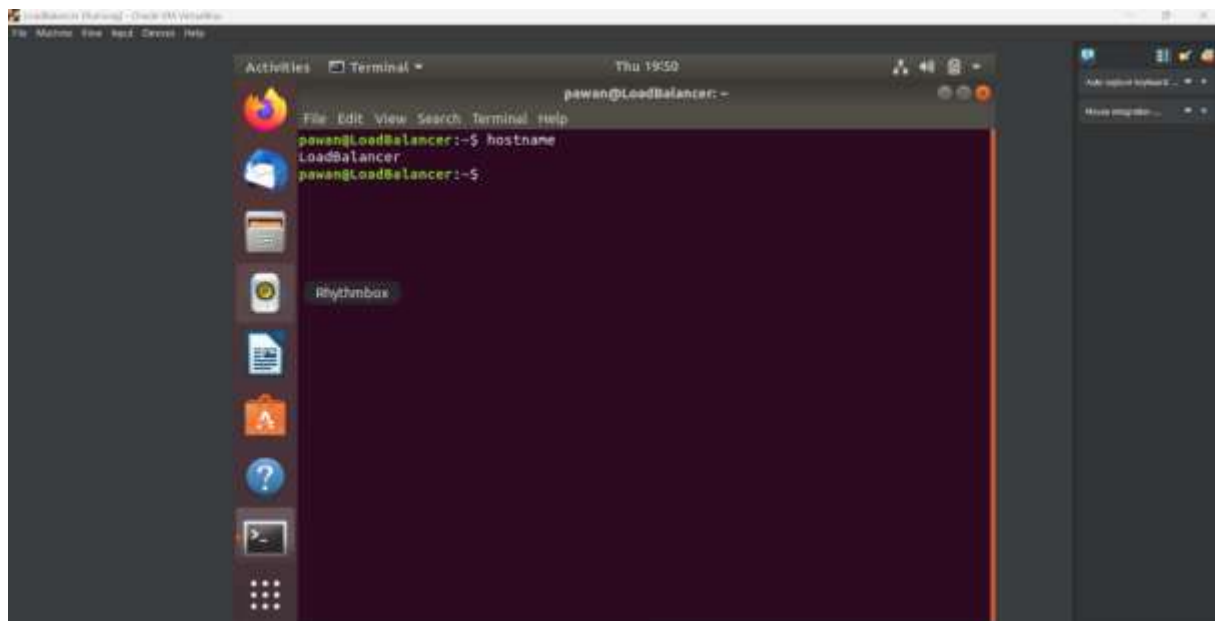


Figure 4: Network configuration showing the IP address of the cloud server.

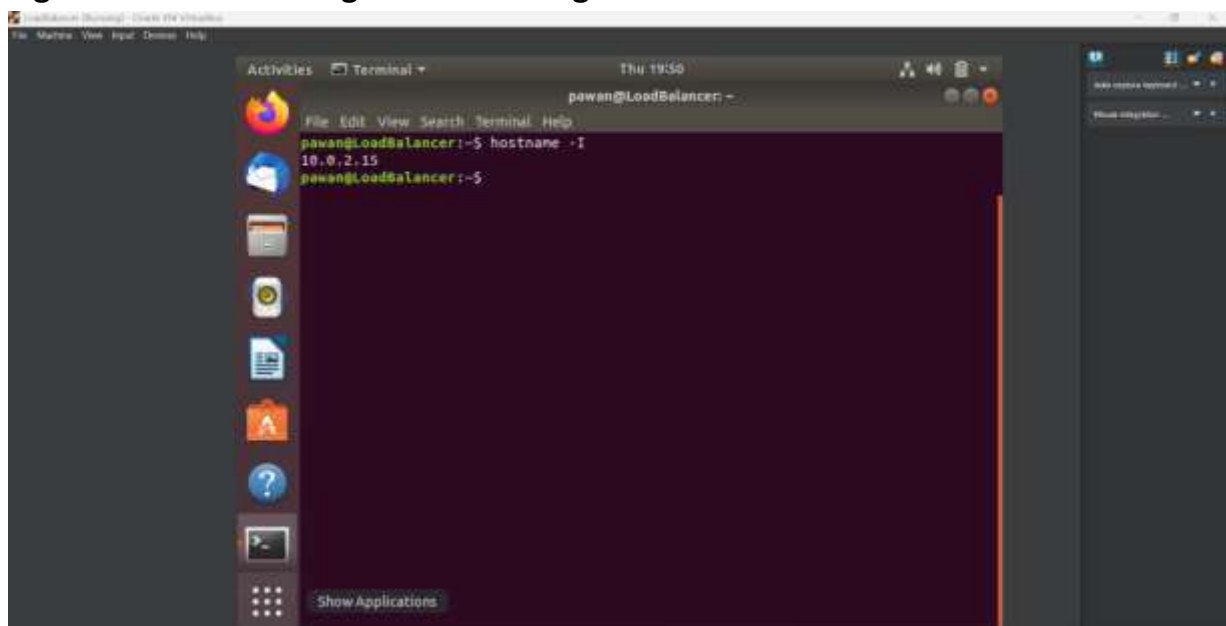


Figure 5: Secure Shell (SSH) connection established successfully.

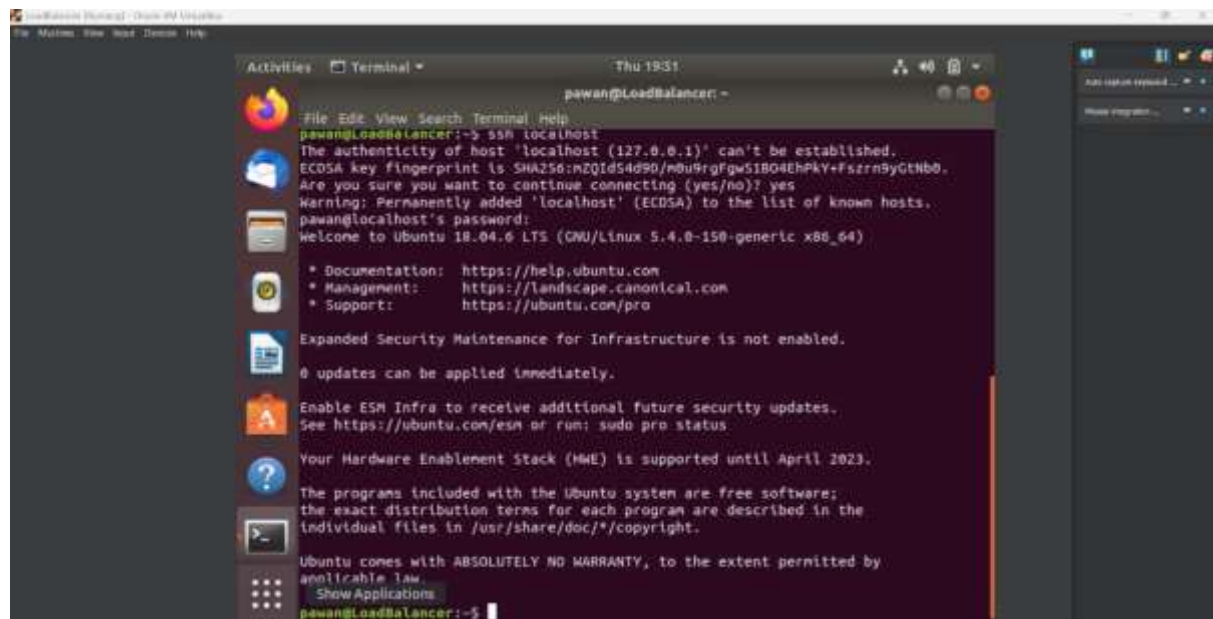
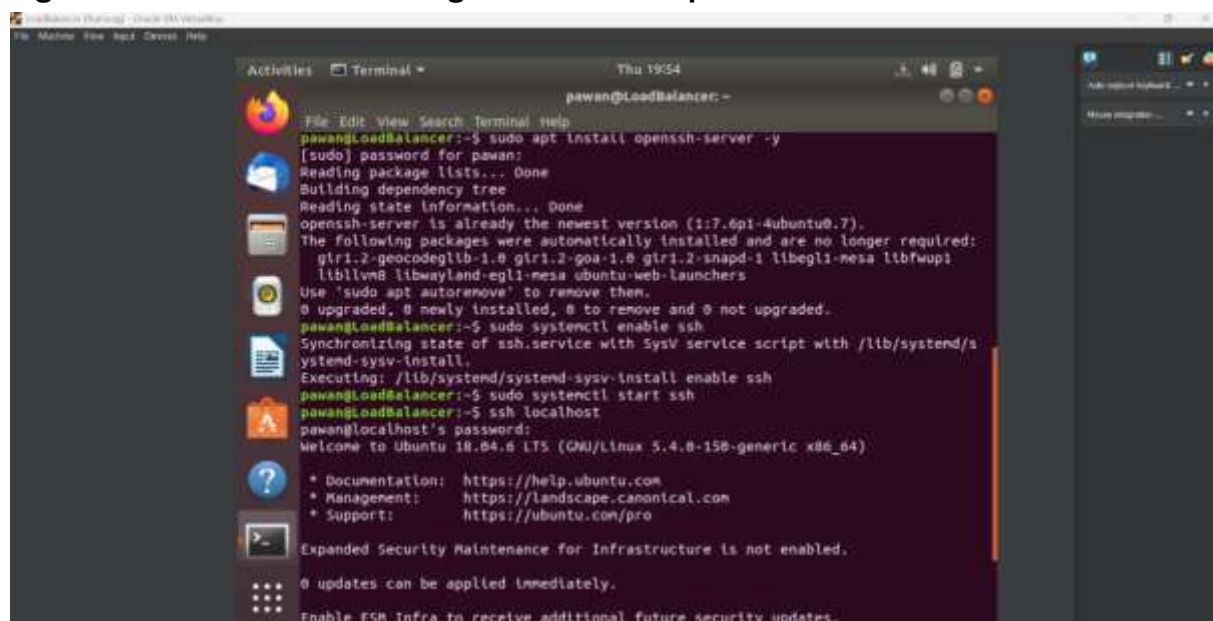


Figure 6: Installation and configuration of the OpenSSH Server.



Result

The hybrid cloud environment was successfully simulated using VirtualBox networking. Secure communication between virtual machines demonstrated hybrid cloud connectivity.

Conclusion

The proposed hybrid cloud architecture provides secure communication between cloud and on-premises environments while maintaining regulatory compliance. Identity management and encryption improve security, while cloud services provide scalability and flexibility.

Question 5: Multi-Region High Availability for a SaaS Application

Marks: 5

Scenario

A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

1. Understanding of the Scenario

A SaaS application must remain available even if an entire cloud region becomes unavailable. This requires multiple deployment regions, replicated databases, automated failover mechanisms, and continuous monitoring to minimize downtime and maintain uninterrupted service.

2. Multi-Region Deployment Strategy

The application is deployed in multiple cloud regions using identical server configurations. Global load balancing distributes traffic to the nearest available region. Application servers remain stateless while user session information is stored in replicated databases. Infrastructure as Code ensures all regions maintain identical configurations.

3. Data Replication & Failover Configuration

Databases are continuously replicated between regions to maintain synchronized data. Automated health checks monitor server availability. If the primary region fails, traffic is automatically redirected to a secondary region. Replication policies ensure minimal data loss, while regular failover testing confirms disaster recovery readiness.

4. Monitoring & Alerting Strategies

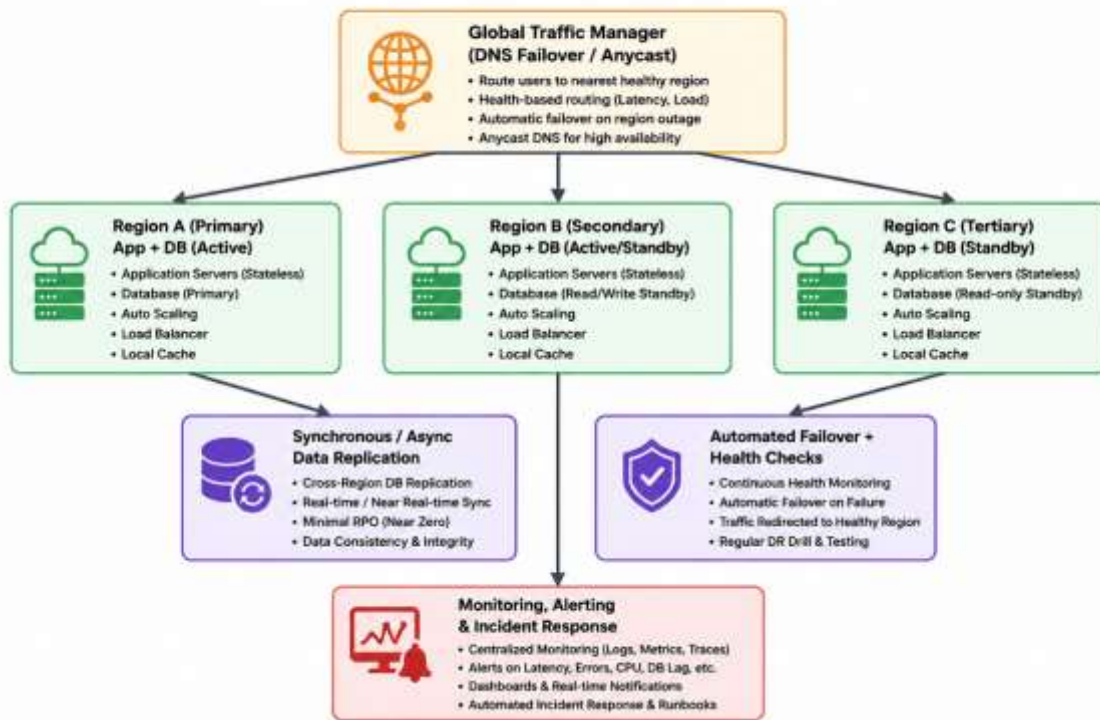
Continuous health checks monitor server performance. Centralized dashboards collect logs, metrics, and performance data from all regions. Alerts notify administrators whenever CPU utilization, latency, or error rates exceed predefined thresholds. Replication lag is monitored continuously to ensure data consistency between regions.

5. Decision Justification & Risks

The proposed architecture provides high availability by eliminating single points of failure. Multiple deployment regions improve fault tolerance and disaster recovery. Continuous monitoring enables early detection of system failures. Potential risks include replication delays, network latency, and increased operational costs, which can be minimized through proper configuration and monitoring.

Proposed Architecture Diagram

Q5: Multi-Region High Availability (SaaS)



Hands-On Demonstration using Oracle VirtualBox

Software Used

- Oracle VirtualBox
- Ubuntu Desktop
- HAProxy
- Nginx

Screenshots

Figure 1: Oracle VirtualBox Manager displaying multiple virtual machines used to simulate a multi-region cloud deployment.

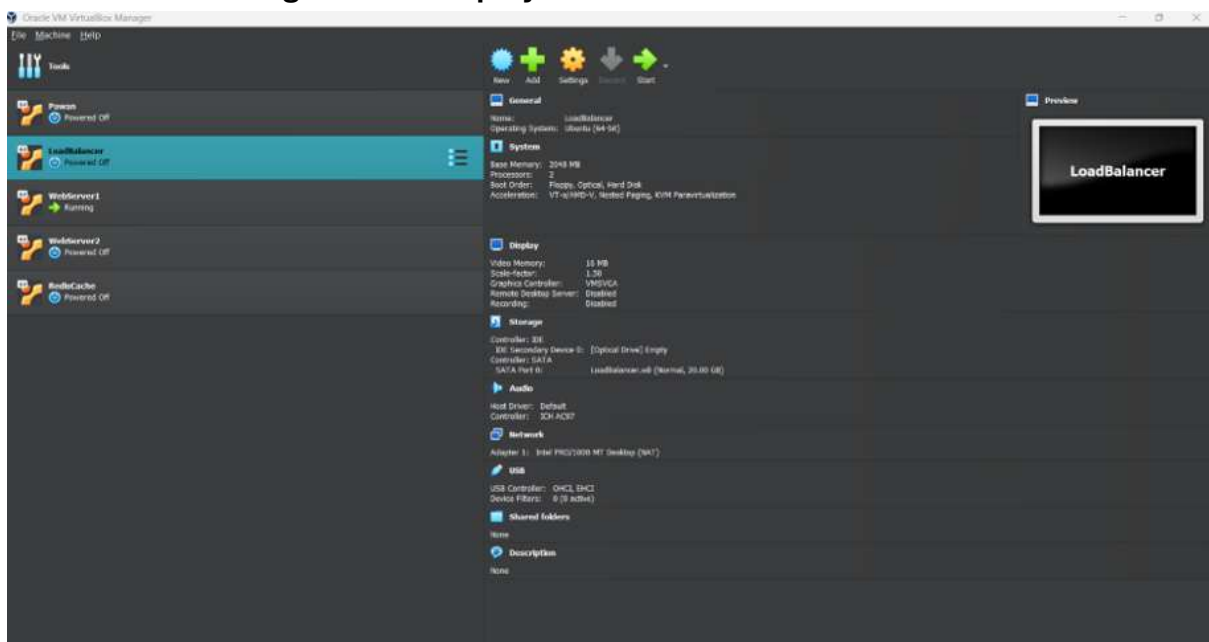
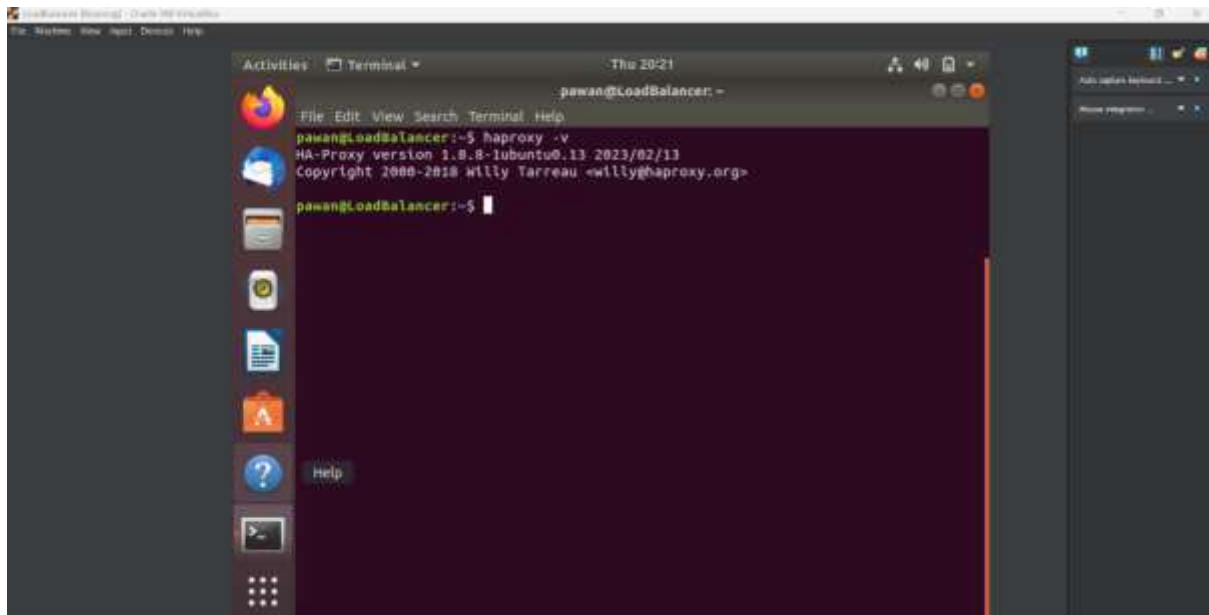


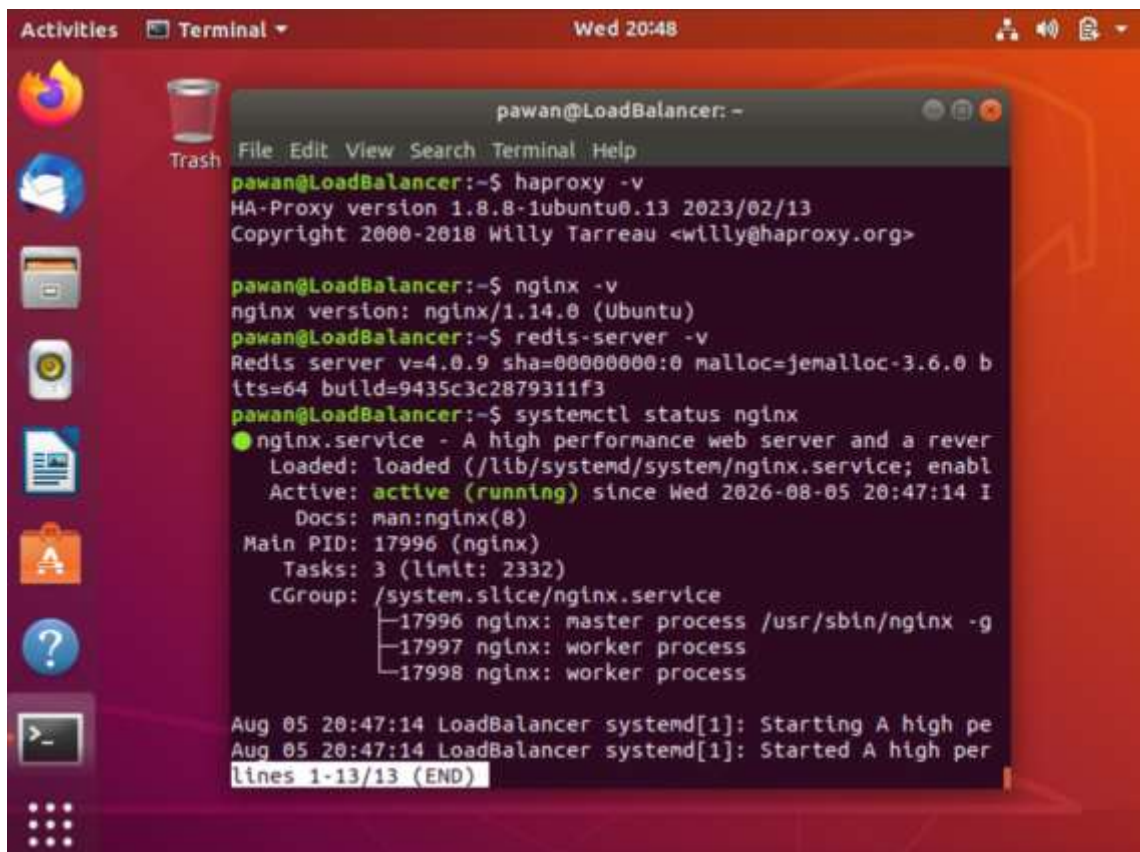
Figure 2: Verification of HAProxy installation.



A terminal window titled 'Terminal' with a dark background. The prompt is 'pawan@LoadBalancer: ~'. The user has entered the command 'haproxy -v', which outputs: 'HA-Proxy version 1.8.8-1ubuntu0.13 2023/02/13 Copyright 2000-2018 Willy Tarreau <willy@haproxy.org>'. The prompt is now 'pawan@LoadBalancer:~\$'.

```
pawan@LoadBalancer: ~  
pawan@LoadBalancer:~$ haproxy -v  
HA-Proxy version 1.8.8-1ubuntu0.13 2023/02/13  
Copyright 2000-2018 Willy Tarreau <willy@haproxy.org>  
pawan@LoadBalancer:~$
```

Figure 3: Nginx web server running on WebServer1.



A terminal window titled 'Terminal' with a dark background. The prompt is 'pawan@LoadBalancer: ~'. The user has entered several commands: 'haproxy -v' (output: 'HA-Proxy version 1.8.8-1ubuntu0.13 2023/02/13 Copyright 2000-2018 Willy Tarreau <willy@haproxy.org>'), 'nginx -v' (output: 'nginx version: nginx/1.14.0 (Ubuntu)'), 'redis-server -v' (output: 'Redis server v=4.0.9 sha=00000000:0 malloc=jemalloc-3.6.0 bits=64 build=9435c3c2879311f3'), and 'systemctl status nginx'. The 'systemctl status nginx' output shows that the 'nginx.service' is 'active (running)' since 'Wed 2026-08-05 20:47:14'. It also lists the main PID as 17996 and three tasks: the master process (PID 17996) and two worker processes (PIDs 17997 and 17998). At the bottom, there are two log messages: 'Aug 05 20:47:14 LoadBalancer systemd[1]: Starting A high pe' and 'Aug 05 20:47:14 LoadBalancer systemd[1]: Started A high per'. The terminal output is highlighted in yellow.

```
pawan@LoadBalancer: ~  
pawan@LoadBalancer:~$ haproxy -v  
HA-Proxy version 1.8.8-1ubuntu0.13 2023/02/13  
Copyright 2000-2018 Willy Tarreau <willy@haproxy.org>  
  
pawan@LoadBalancer:~$ nginx -v  
nginx version: nginx/1.14.0 (Ubuntu)  
pawan@LoadBalancer:~$ redis-server -v  
Redis server v=4.0.9 sha=00000000:0 malloc=jemalloc-3.6.0 bits=64 build=9435c3c2879311f3  
pawan@LoadBalancer:~$ systemctl status nginx  
● nginx.service - A high performance web server and a rever  
   Loaded: loaded (/lib/systemd/system/nginx.service; enabl  
   Active: active (running) since Wed 2026-08-05 20:47:14 I  
     Docs: man:nginx(8)  
    Main PID: 17996 (nginx)  
      Tasks: 3 (limit: 2332)  
     CGroup: /system.slice/nginx.service  
             └─17996 nginx: master process /usr/sbin/nginx -g  
               └─17997 nginx: worker process  
                 └─17998 nginx: worker process  
  
Aug 05 20:47:14 LoadBalancer systemd[1]: Starting A high pe  
Aug 05 20:47:14 LoadBalancer systemd[1]: Started A high per  
lines 1-13/13 (END)
```

Figure 4: Nginx web server running on WebServer2.

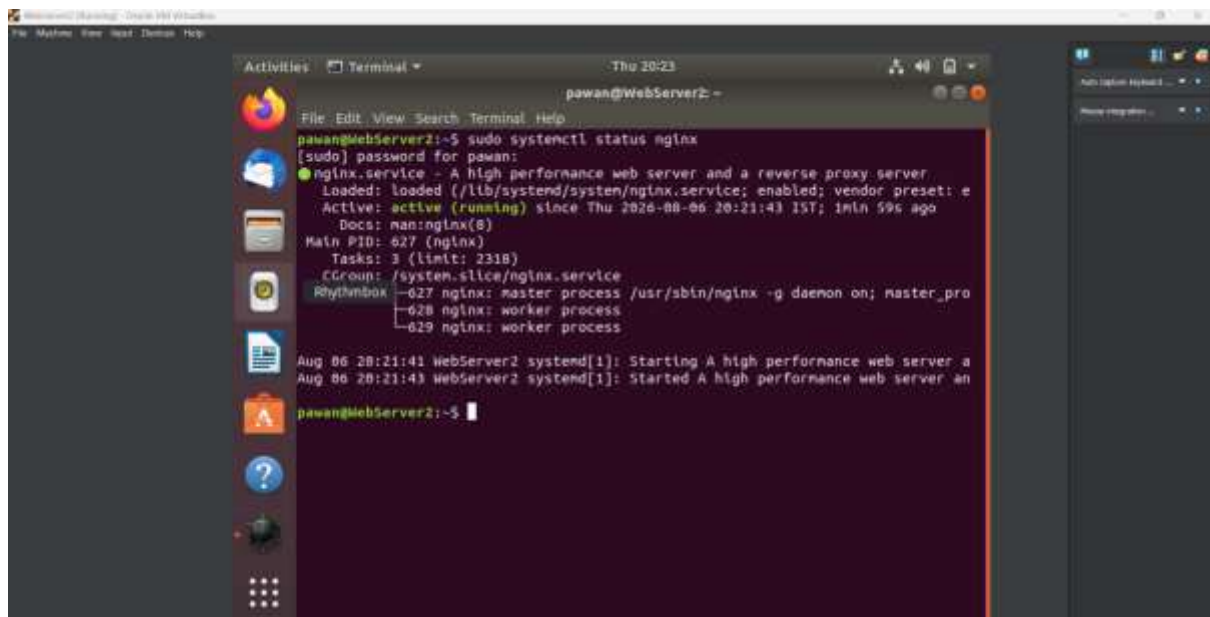


Figure 5: Multiple virtual machines representing different cloud regions.

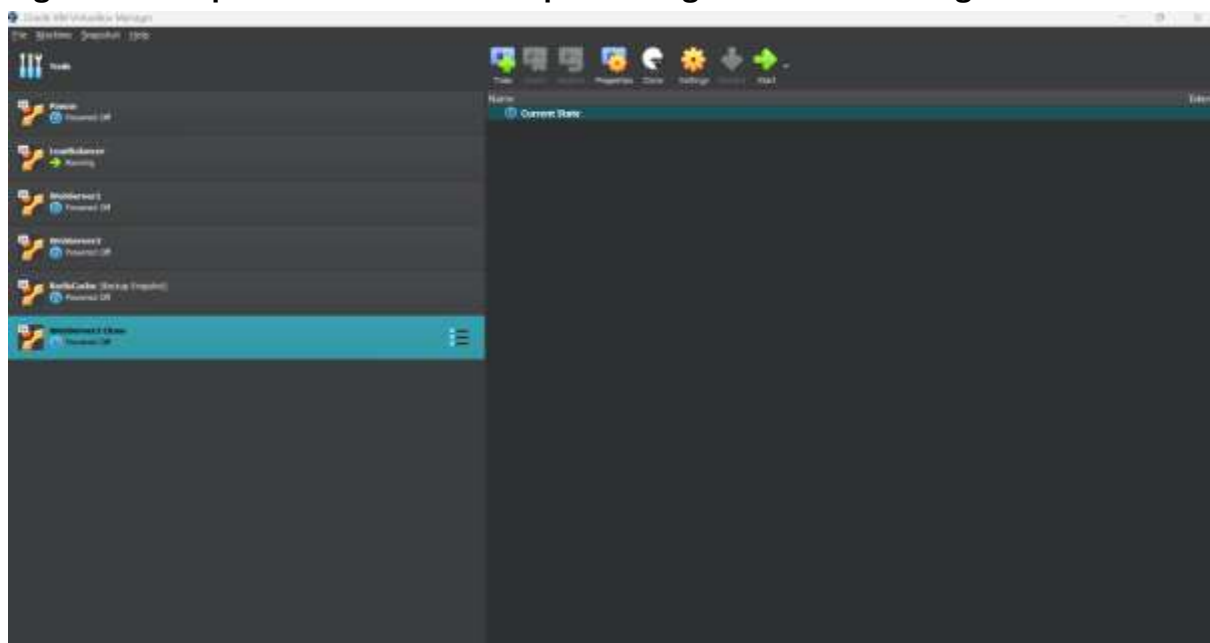


Figure 6: Browser accessing the application through the load balance

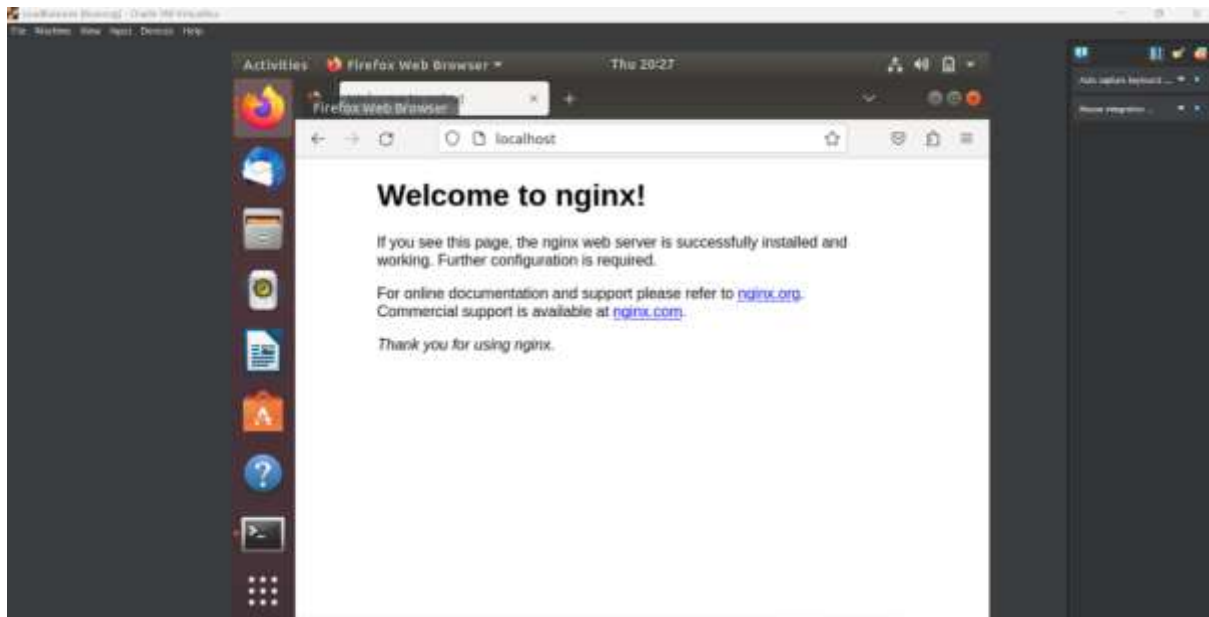
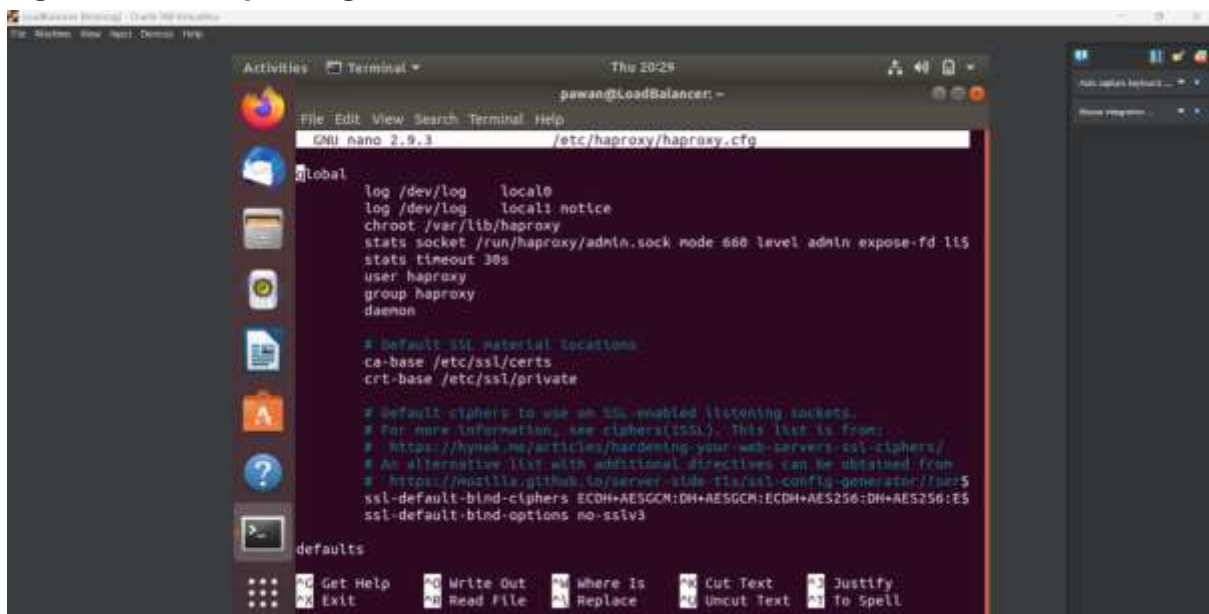


Figure 7: HAProxy configuration file.



Result

The multi-region high availability architecture was successfully demonstrated using VirtualBox. Multiple virtual machines simulated different cloud regions, while load balancing and replication concepts ensured continuous service availability.

Conclusion

The proposed multi-region architecture improves application availability, fault tolerance, and disaster recovery. Automated failover, data replication, and continuous monitoring provide reliable cloud services even during regional failures. The VirtualBox implementation successfully demonstrates the concepts of high availability in cloud computing.